STOCK MOVEMENT PREDICTION USING HYBRID MACHINE LEARNING MODELS

A Project Report

Submitted in the partial fulfilment of the requirements for the

award of the degree

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)

Submitted By

K. ALEKHYA	**(228A1A4207)**
M. YOGANJALI	**(228A1A4212)**
A. AKHILA	**(228A1A4201)**
A. SASHI KIRAN	**(228A1A4231)**

Under the Esteemed Guidance

Mrs. DR. P. KALYANI

(Associate Professor)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)

RISEKRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS

Valluru-523272

(Affiliated to JNTU, Kakinada & Approved by AICTE, New Delhi)

Accredited by NBA, NAAC with 'A' certified by ISO9001:2015

2024-2025

RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS

Valluru – 523272

(Affiliated to JNTU, Kakinada & Approved by AICTE, New Delhi)

Accredited by NBA, NAAC with 'A' certified by ISO9001:2015

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)

BONAFIDE CERTIFICATE

This is to certify that this project work entitled **"STOCK MOVEMENT PREDICTION USING HYBRID MACHINE LEARNING MODELS"** is being submitted by **K.ALEKHYA (228A1A4207), M.YOGNAJALI (228A1A4212), A.AKHILA (228A1A4201), A.SASHI KIRAN (228A1A4231)** in partial fulfillment of the requirements for degree of Bachelor of Technology in COMPUTER SCIENCE & ENGINEERING – AI&ML from JNTUK, Kakinada during the period of 2025- 26 is a record of Bonafide work carried out by them under our esteemed guidance and supervision.

PROJECT GUIDE
Mrs. DR. P. KALYANI
Assistant professor

HEAD OF THE DEPARTMENT
DR. G. BHARGAVI
Professor & HOD

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the work is being presented in this dissertation entitled

"STOCK MOVEMENT PREDICTION USING HYBRID MACHINE LEARNING MODELS" submitted towards the partial fulfilment of requirements for the award of the degree of Bachelor of Technology in COMPUTER SCIENCE & ENGINEERING – AI&ML, work carried out under the supervision of Mrs. DR. P. KALYANI, Assistant Professor, RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS, Valluru, Ongole. The results embodied in this dissertation report have not been submitted to any other University for the award of any other degree. Furthermore, the technical details furnished in various chapters of this report are purely relevant to the above project and there is no deviation from the theoretical point of view for design, development and implementation.

K. Alekhya	**228A1A4207**
M. Yoganjali	**228A1A4212**
A. Akhila	**228A1A4201**
A. Sashi Kiran	**228A1A4231**

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our guide **Mrs. Dr. P. Kalyani ,** Assistant Professor, for providing her valuable guidance, comments, suggestions, and support throughout the course of the project.

We express our heartful thanks to **Dr. G. Bhargavi,** Head of the Department, **COMPUTER SCIENCE & ENGINEERING – AI&ML**, for her valuable support to bring out this project in time.

We pay our profound sense of gratitude to our Principal **Prof. Dr A.V. Bhaskara Rao** for providing an excellent environment in our college and helping us at all points for achieving our task.

We would like to express our sincere thanks to the Management of **RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS, Valluru** for providing a good environment and infrastructure.

Finally, We thank all our faculty members, supporting staff of CSE – AI&ML Department and friends for their kind co-operation and valuable help for the completion of the project.

Project Associates

K. ALEKHYA 228A1A4207 M. YOGNAJALI 228A1A4212 A. AKHILA 228A1A4201 A. SASHI KIRAN 228A1A4231

RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS

Valluru – 523272

(Affiliated to JNTU, Kakinada & Approved by AICTE, New Delhi)

Accredited by NBA, NAAC with 'A' certified by ISO 9001:2015

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)

Vision of the Institute	To be a premier institution in technical education by creating professionals of global standards with ethics and social responsibility for the development of the nation and the mankind.
Mission of the Institute	Impart Outcome Based Education through well qualified and educated faculty.
	Provide state-of-the-art infrastructure and facilities for application oriented research.
	Reinforce technical skills with life skills and entrepreneurship skills.
	Promote cutting-edge technologies to produce industry-ready professionals.
	Facilitate interaction with all stakeholders to foster ideas and innovation.
	Inculcate moral values, professional ethics and social responsibility
Vision of the Department	To be a center of excellence in computer science and engineering for value based education to serve humanity and contribute for socio-economic development.

Mission of the Department	Provide professional knowledge by student centric teaching-learn in process to contribute to software industry.
	Inculcate training on cutting edge technologies for industry needs.
	Create an academic ambience leading to research.
	Promote industry institute interaction for real time problem solving

Program Outcomes (POs)

PO1	**Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering Problems
PO2	**Problem Analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	**Design/Development of Solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and Environmental considerations.
PO4	**Conduct Investigations of Complex Problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information provide valid conclusions.
PO5	**Modern Tool Usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex Engineering activities with an understanding of the limitations.
PO6	**The Engineer and Society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the Professional engineering practice.
PO7	**Environment and Sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the Knowledge of, and need for sustainable development.
PO8	**Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice
PO9	**Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multi-disciplinary settings.

PO10	**Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions
PO11	**Project Management and Finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments
PO12	**Life-long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Educational Objectives (PEOs):

PEO1:	Develop software solutions for real world problems by applying Mathematics
PEO2:	Function as members of multi-disciplinary teams and to communicate effectively using modern tools.
PEO3:	Pursue career in software industry or higher studies with continuous learning and apply professional knowledge.
PEO4:	Practice the profession with ethics, integrity, leadership and social responsibility.

Program Specific Outcomes (PSOs):

PSO1	**Domain Knowledge:** Apply the Knowledge of Programming Languages, Networks and Databases for design and development of Software Applications.
PSO2	**Computing Paradigms:** Understand the evolutionary changes in computing possess knowledge of context aware applicability of paradigms and meet the challenges of the future.

RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS

Valluru – 523272

(Affiliated to JNTU, Kakinada & Approved by AICTE, New Delhi)

Accredited by NBA, NAAC with 'A' certified by ISO 9001:2015

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence & Machine Learning)

Name of the Course : Project Year & Semester : IV Year II

Academic Year : 2025-2026 Regulation : R20

Course Outcomes:

Course Outcomes (COs)		Taxonomy Level
C421.1	Develop application for community needs	Applying
C421.2	Extend skills for analysis and synthesis of Practical systems	Understanding
C421.3	Get hold of the use of new tools efficiently and creatively.	Analyzing
C421.4	Work in team to carry out analysis and cost-effective, environmentally friendly designs of engineering systems.	Analyzing
C421.5	Write Technical/Project reports and oral presentation of the work done to an audience.	Applying
C421.6	Demonstrate a product developed.	Understanding

PROJECT COORDINATOR **HEAD OF THE DEPARTMENT**

CO Vs PO Mapping

Course Outcomes (COs)	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
C421.1	2	2	3	3	3	3	2	2	3	2	2	2
C421.2	2	2	3	3	3	3	2	2	3	3	3	3
C421.3	2	2	3	3	3	3	2	2	3	2	2	2
C421.4	2	2	3	3	3	3	3	3	3	3	3	3
C421.5	2	2	3	2	3	3	2	3	3	3	3	3
C421.6	2	2	2	2	3	3	3	3	3	3	3	3
C421	**2.00**	**2.00**	**2.83**	**2.67**	**3.00**	**3.00**	**2.33**	**2.50**	**3.00**	**2.67**	**2.67**	**2.67**

CO Vs PSO Mapping

Course out Comes (Cos)	PSO 1	PSO 2
C421.1	3	3
C421.2	3	3
C421.3	3	3
C421.4	3	3
C421.5	2	2
C421.6	3	3
	2.83	2.83

1: Low 2: Medium 3: High

Guide Signature

STOCK MOVEMENT PREDICTION USING HYBRID MACHINE LEARNING MODELS

INDEX

LIST OF FIGURES

ABSTRACT:

This paper examines a hybrid model which combines a K-Nearest Neighbors (KNN) approach with a probabilistic method for the prediction of stock price trends. One of the main problems of KNN classification is the assumptions implied by distance functions. The assumptions focus on the nearest neighbors which are at the centroid of data points for test instances. This approach excludes the non-centric data points which can be statistically significant in the problem of predicting the stock price trends. For this it is necessary to construct an enhanced model that integrates KNN with a probabilistic method which utilizes both centric and non-centric data points in the computations of probabilities for the target instances. The embedded probabilistic method is derived from Bayes' theorem. The prediction outcome is based on a joint probability where the likelihood of the event of the nearest neighbors and the event of prior probability occurring together and at the same point in time where they are calculated. The proposed hybrid KNN Probabilistic model was compared with the standard classifiers that include KNN, Naive Bayes, One Rule (One R) and Zero Rule (ZeroR). The test results showed that the proposed model outperformed the standard classifiers which were used for the comparisons.

Keywords: Stock Price Prediction, K-Nearest Neighbors, Bayes' Theorem, Naïve Bayes, Probabilistic Method.

CHAPTER-1
INTRODUCTION

INTRODUCTION

1.1 Introduction

Analyzing financial data in securities has been an important and challenging issue in the investment community. Stock price efficiency for public listed firms is difficult to achieve due to the opposing effects of information competition among major investors and the adverse selection costs imposed by their information advantage.

There are two main schools of thought in analyzing the financial markets. The first approach is known as fundamental analysis. The methodology used in fundamental analysis.

This approach examines a company's financial reports, management, industry,micro and macro-economic factors.

The second approach is known as technical analysis. The methodology used in technical analysis for forecasting the direction of prices is through the study of historical market data. Technical analysis uses a variety of charts to anticipate what are likely to happen. The stock charts include candlestick charts, line charts, bar charts, point and figure charts, OHLC (open-high-low-close) charts and mountain charts. The charts are viewable in different time frames with price and volume. There are many types of indicators used in the charts, including resistance, support, breakout, trending and momentum.

Several alternatives to approach this type of problem have been proposed, which range from traditional statistical modelling to methods based on computational intelligence and machine learning. Vanstone and Tan surveyed the works in the domain of applying soft computing to financial trading and investment. They categorized the papers reviewed in the following areas: time series, optimization, hybrid methods, pattern recognition and classification. Within the context of financial trading discipline, the survey showed that most of the research was being conducted in the field of technical analysis. An integrated fundamental and technical analysis model was examined to evaluate the stock price trends by focusing on macro-economic analysis. It also analyzed the company behaviour and the

associated industry in relation to the economy which in turn provide more information for

investors in their investment decisions.

A nearest neighbor search (NNS) method produced an intended result by the use of

KNN technique with technical analysis. This model applied technical analysis on stock

market data which include historical price and trading volume. It applied technical indicators

made up of stop loss, stop gain and RSI filters. The KNN algorithm part applied the distance

function on the collected data. This model was compared with the buy-and-hold strategy by

using the fundamental analysis approach.

Fast Library for Approximate Nearest Neighbors (FLANN) is used to perform the

searches for choosing the best algorithm found to work best among a collection of algorithms

in its library. Majhi etal. examined the FLANN model to predict the S&P 500 indices, and

the FLANN model was established by performing fast approximate nearest neighbor searches

in high dimensional spaces. A hybrid intelligent data mining methodology based on Genetic Algorithm - Support

Vector Machine Model was reviewed to explore stock market tendency. This approach make.

use of the genetic algorithm for variable selection in order to improve the speed of support

vector machine by reducing the model complexity, and then the historical data is used to

identify stock market trends. Hybrid techniques can be used to improve the existing

forecasting models due to the limitation of ANN like black box technique. A combination of

methods such as fuzzy rule-based system, fuzzy neural network and Kalman filter with

hybrid neuro-fuzzy architecture have been developed to predict financial time series data.

This research studies a hybrid approach through the use of KNN algorithm and a

probabilistic method for predicting the stock price trends.

1.2 Comparison of various learning classifiers

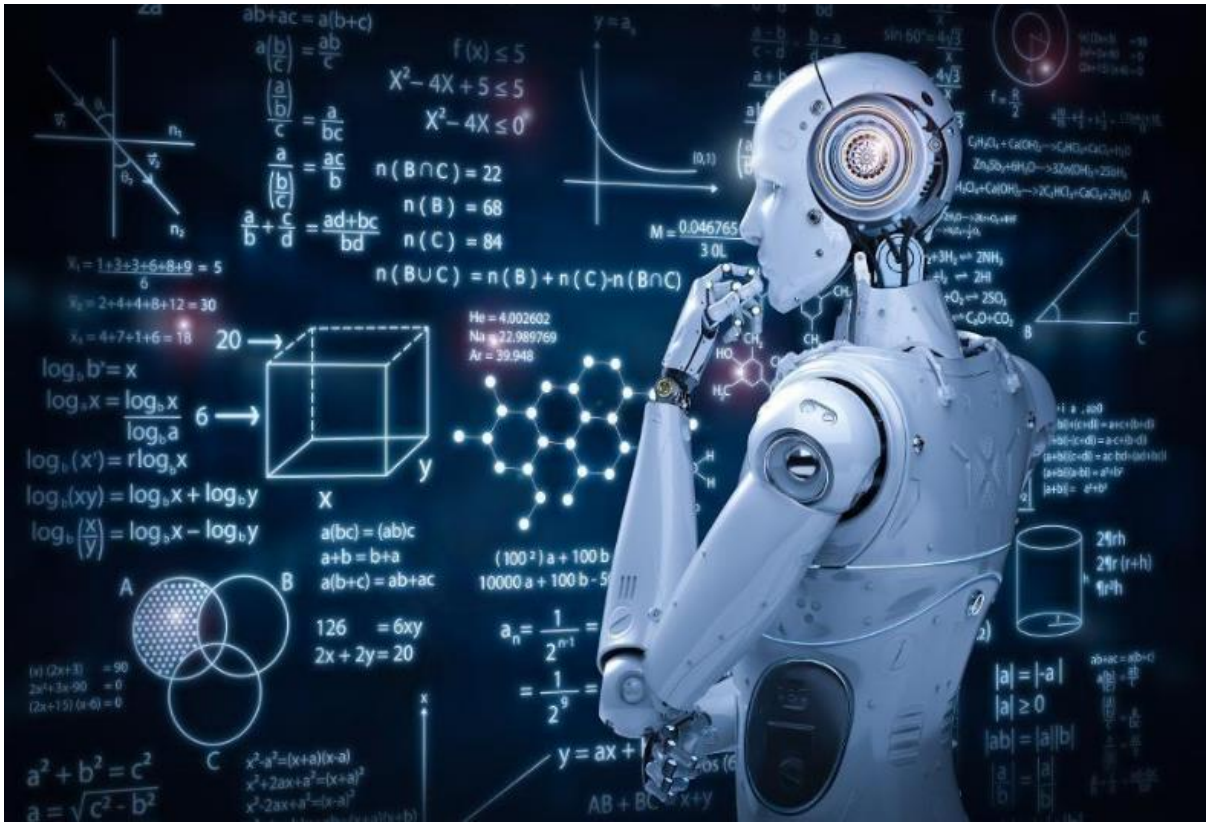
When developing a classifier using various functions from different classifiers, it is important to compare the performances of the classifiers. Simulation results can provide us with direct comparison results for the classifiers with a statistical analysis of the objective functions. The hybrid KNN-Probabilistic model was compared with the supervised learning and classification algorithms, including 'KNN', 'Naïve Bayes', 'OneR' and 'ZeroR'.

Machine Learning (ML):

Introduction

Machine learning is a subfield of artificial intelligence (AI). The goal of machine learning generally is to understand the structure of data and fit that data into models that can be understood and utilized by people. Although machine learning is a field within computer science, it differs from traditional computational approaches. In traditional computing, algorithms are sets of explicitly programmed instructions used by computers to calculate or problem solve.

Machine learning algorithms instead allow for computers to train on data inputs and use statistical analysis in order to output values that fall within a specific range. Because of this, machine learning facilitates computers in building models from sample data in order to automate decision-making processes based on data inputs.



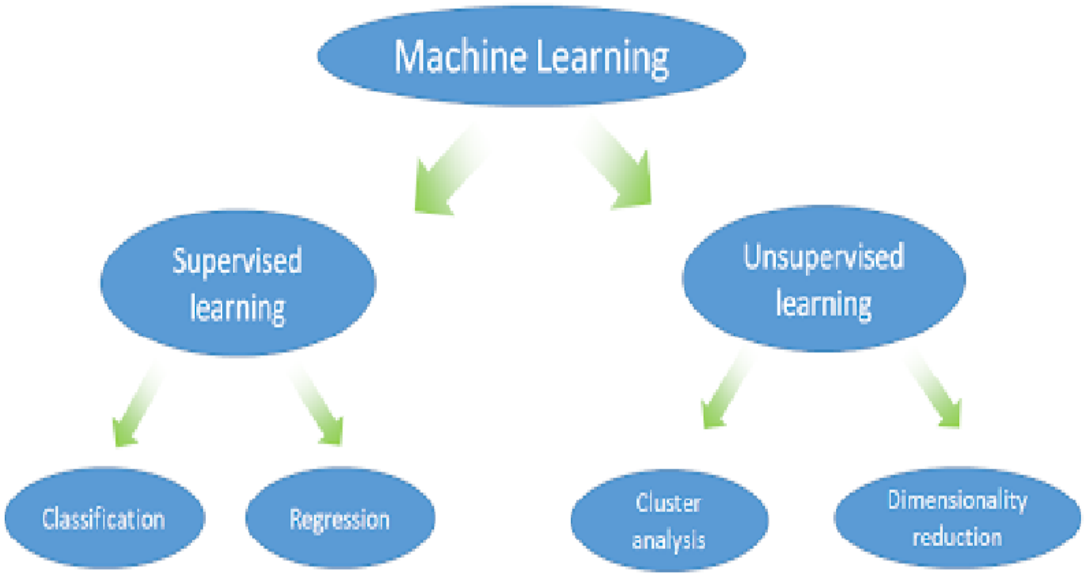
Any technology user today has benefitted from machine learning. Facial recognition technology allows social media platforms to help users tag and share photos of friends. Optical character recognition (OCR) technology converts images of text into movable type. Machine learning is a continuously developing field. Because of this, there are some considerations to keep in mind as you work with machine learning methodologies, or analyze the impact of machine learning processes. We'll look into the common machine learning methods of supervised and unsupervised learning, and common algorithmic approaches in machine learning, including the k-nearest neighbor algorithm, decision tree learning, and deep learning.

Machine Learning Methods:

Two of the most widely adopted machine learning methods are

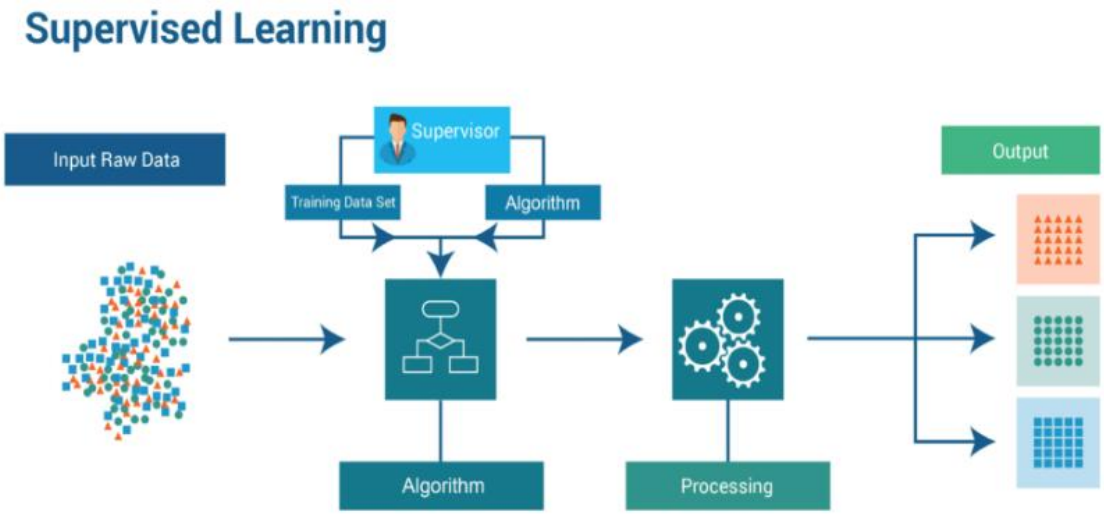
1. Supervised learning

2. Unsupervised learning



1. Supervised learning:

In supervised learning, the computer is provided with example inputs that are labeled with their desired outputs. The purpose of this method is for the algorithm to be able to "learn" by comparing its actual output with the "taught" outputs to find errors, and modify the model accordingly. Supervised learning therefore uses patterns to predict label values on additional unlabeled data.

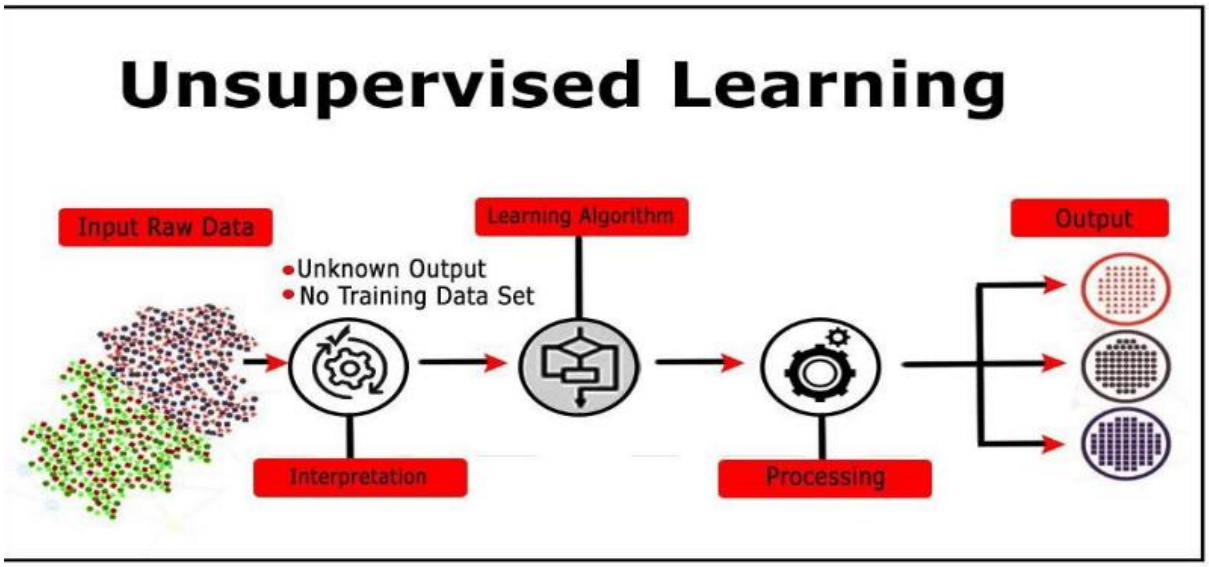


A common use case of supervised learning is to use historical data to predict statistically likely future events. It may use historical stock market information to anticipate upcoming fluctuations, or be employed to filter out spam emails. In supervised learning, tagged photos of dogs can be used as input data to classify untagged photos of dogs.

2. Unsupervised Learning:

In unsupervised learning, data is unlabeled, so the learning algorithm is left to find commonalities among its input data. As unlabeled data are more abundant than labeled data, machine learning methods that facilitate unsupervised learning are particularly valuable.

The goal of unsupervised learning may be as straightforward as discovering hidden patterns within a dataset, but it may also have a goal of feature learning, which allows the computational machine to automatically discover the representations that are needed to classify raw data. Unsupervised learning is commonly used for transactional data.



Without being told a “correct” answer, unsupervised learning methods can look at complex data that is more expansive and seemingly unrelated in order to organize it in potentially meaningful ways. Unsupervised learning is often used for anomaly detection including for fraudulent credit card purchases, and recommender systems that recommend what products to buy next.

Application of Machine Learning:

➢ Face Recognition

➢ Social Media Services

➢ Virtual Personal Assistants

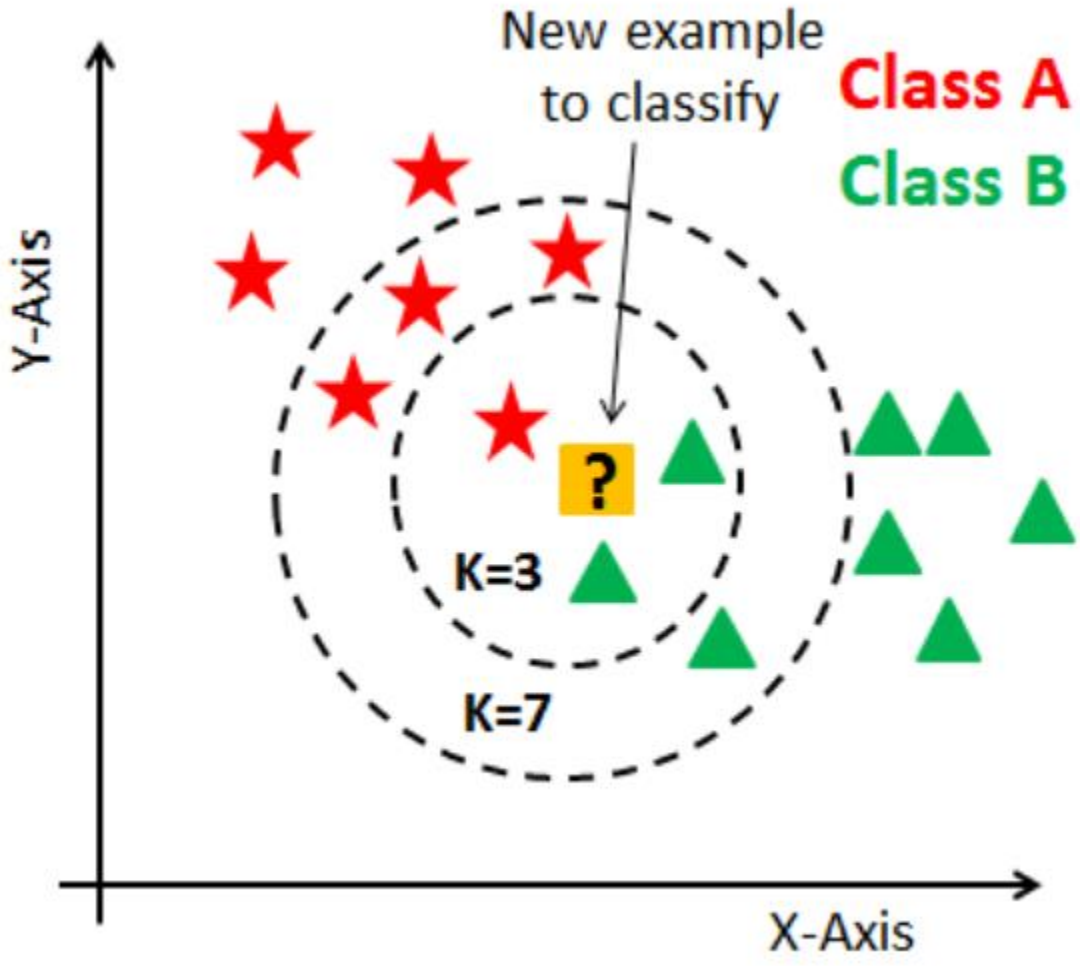
➢ Online Fraud Detection

➢ Product Recommendations

➢ Autonomous

1.2.1 K-Nearest Neighbors Classifier

The KNN algorithm is used to measure the distance between the given test instance and all the instances in the data set, this is done by choosing the 'k' closest instances and then predict the class value based on these nearest neighbors. The 'k' is assigned as number of neighbors voting on the test instance. As such KNN is often referred to as case based learning or an instance-based learning where each training instance is a case from the problem domain. KNN is also referred to as a lazy learning algorithm due to the fact that there is no learning of the model required and all of the computation works happen at the time a prediction is requested. KNN is a non-parametric machine learning algorithm as it makes no assumptions about the functional form of the problem being solved. Each prediction is made for a new instance (x) by searching through the entire training set for the 'k' most nearest instances and applying majority voting rule to determine the prediction outcome.1.4 Applications



A variety of distance functions are available in KNN which include Euclidean, Manhattan,

Minkowski and Hamming. The Euclidean distance function is probably the most commonly used in any distance-based algorithm. It is defined as

$d(x,y) = \sqrt{\sum_{i=1}^{k} (x_i - y_i)^2}$

(1) Where, x and y are two data vectors and k is the number of attributes. The Manhattan distance function is defined as

$d(x,y) = \sum_{i=1}^{k} |x_i - y_i|$

(2) The Minkowski distance function is defined as

$d(x,y) = \left(\sum_{i=1}^{k} (|x_i - y_i|)^p\right)^{1/k}$

(3) The distance functions for Euclidean, Manhattan and Minkowski are used for numerical attributes. The Hamming distance function is defined as

$d(x,y) = \sum_{i=1}^{k} |x_i - y_i|$

x = y => D = 0

x ≠ y => D = 1

(4) Hamming distance is usually used for categorical attributes. The Hamming distance between two data vectors is the number of attributes in which they differ.

Advantages of KNN Algorithm:

• It is simple to implement.

• It is robust to the noisy training data

• It can be more effective if the training data is large.

Disadvantages of KNN Algorithm:

• Always needs to determine the value of K which may be complex some time.

• The computation cost is high because of calculating the distance between the data points for all the training samples.

1.2.2 Bayes' Theorem

The Bayes' theorem plays an important role in probabilistic learning and classification. The Bayesian classification represents a supervised learning method as well as

a statistical method for classification. It has learning and classification methods based on probability theory.

The Bayesian classification is named after Thomas Bayes, who proposed the Bayes' theorem. Bayes' theorem is often called Bayes' rule. The Bayes' rule uses prior probability of each category given no information about an item. Bayesian classification provides probabilistic methods where prior knowledge and observed data can be combined. It has a useful perspective for evaluating a variety of learning algorithms.

The Bayesian classification calculates explicit probabilities for hypothesis and it is also robust to noise in input data. Given a hypothesis h and data D which bears on the hypothesis, Bayes' theorem is stated as

$$\mathrm{P(h/D)} = \frac{P(h/D)P(h)}{P(D)}$$

Where:

P(D): independent probability of D

P(h): independent probability of h: prior probability

P(h|D): conditional probability of h given D: posterior probability

P(D|h): conditional probability of D given h: likelihood

a. Naïve Bayes Classifier

The Naïve Bayes classifier is a classification method based on Bayes' theorem with independence assumptions among predictors. A Naive Bayes classifier assumes that the presence of a particular attribute in a class is unrelated to the presence of any other attribute. A Naive Bayes model is easy to build, with no complicated iterative parameter estimation that makes it particularly useful for very large data sets. In spite of its simplicity, the Naïve Bayes classifier often does surprisingly well and is widely used due to the fact that it often outperforms other more sophisticated classification techniques. The Bayes' rule from (5) can also be expressed as

P(h/D) = P(d1|h) x P(d2|h) x … P(dn|h) x P(h)

Where:

h1, h2, ..., hn be a set of mutually exclusive events that together form the sample space S.

D be any event from the same sample space, such that P(D) > 0.

Advantages of Naïve Bayes Classifier:

• Naïve Bayes is one of the fast and easy ML algorithms to predict a class of datasets.

• It can be used for Binary as well as Multi-class Classifications.

• It performs well in Multi-class predictions as compared to the other Algorithms.

• It is the most popular choice for text classification problems.

Disadvantages of Naïve Bayes Classifier:

• Naive Bayes assumes that all features are independent or unrelated, so it cannot learn the relationship between features.

b. Bayesian Network Classifier

A Bayesian network is a type of graph which is used to model events for probabilistic relationships among a set of random variables. This can then be used for inference. The graph is called a directed acyclic graph (DAG). DAG contains nodes and edges. A node is also called a vertex. The nodes of the graph represent random variables. Edges represent the connections between the nodes. If two nodes are connected by an edge, it has an associated probability that it will transmit from one node to the other. The graph is directed and does not contain any cycles. A directed graph has ordered pairs of vertices and it consists of a set of vertices and a set of arcs. In a DAG diagram, a vertex is represented by a circle with a label, and an edge is represented by an arrow or line extending from one vertex to another.
Bayesian Network provides a representation of the joint probability distribution over the variables. A problem domain is modeled by a list of variables X1, …, X n. Knowledge about the problem domain is represented by a joint probability P(X1, …, Xn).
A joint probability refers to the probability of more than one variable occurring together, such as the probability of event A and event B. Notation for joint probability of two events takes the form P(A∩B) which denotes the probability of the intersection of A and B. Joint probability for dependent events of event A and event B can be expressed as

$$P(A \cap B) = P(A) \times P(B \mid A)$$

Where:

"∩" denotes the point where A and B intersect.

P(A) is the probability of event A occurs

P(B |A) is the conditional probability of B given A

Joint probability for independent events of event A and event B can be expressed as

$P(A \cap B) = P(A) \times P(B)$

In a joint distribution for independent variables, two discrete random variables X and

Y are independent if the joint probability mass function conforms to

$P(X = x \text{ and } Y = y) = P(X = x) \times P(Y = y)$ for all x and y.

1.2.3 Decision Tree Learning

For general use, decision trees are employed to visually represent decisions and show or inform decision making. When working with machine learning and data mining, decisiontrees are used as a predictive model. These models map observations about data to conclusions about the data's target value.The goal of decision tree learning is to create a model that will predict the value of a target based on input variables.In the predictive model, the data's attributes that are determined through observation are represented by the branches, while the conclusions about the data's target value are represented in the leaves.

When "learning" a tree, the source data is divided into subsets based on an attribute value test, which is repeated on each of the derived subsets recursively. Once the subset at a node has the equivalent value as its target value has, the recursion process will be complete.

Let's look at an example of various conditions that can determine whether or not someone should go fishing. This includes weather conditions as well as barometric pressure conditions.

1.3 Related Works

A study on a KNN approach that made use of economic indicators and classification techniques to predict the stock price trends yielded considerable precision. Four indicators were identified and calculated based on the technical indicators and their formulas. The values of the indicators were normalized in the range between -1 and +1. Accuracy and F-measure were calculated to evaluate the performance of the model. Calculation of these evaluation measures required estimating Recall and Precision which were assessed from True Negative (TN), False Negative (FN), True Positive (TP) and False Positive (FP). The performance of the KNN model was improved by using the optimal value for the k parameter. The evaluation of the KNN model and its performance is illustrated in Table 1.

Table 1: KNN Model Performance.

Measurement	K Parameter		
	25	45	70
Accuracy	0.8138	0.8059	0.8132
F-measure	0.8202	0.8135	0.8190

Another study examined a model named Integrated Multiple Linear Regression-One Rule Classification Model (Regression-OneR) that predicts the stock class outputs in classification form based on the initial predicted outputs in regression form. The regression classifiers were used to predict the outputs in continuous values as opposed to the classification classifiers which were used to predict the outputs in categorical values. As illustrated in Table 2, the result showed that the hybrid regression-classification approach produced better accuracy rate and lower Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) as compared with the model which incorporated only a standard classification algorithm.

Table 2: Prediction Results of Classifiers.

Classifier	Accuracy	MAE	RMSE
Regression-OneR	85.0746	0.1493	0.3863
OneR	71.6418	0.2836	0.5325
ZeroR	64.1791	0.4619	0.4805
J48 Decision Tree	82.0896	0.2121	0.3796
REP Tree	76.1194	0.2764	0.4207

CHAPTER-2
LITERATURE SURVEY

LITERATURE SURVEY

Recent Studies

1. "Machine Learning for Earthquake Early Warning" - Journal of Seismology - Authors:

   Y. Wu et al.

   - Method: Convolutional Neural Networks (CNNs) for seismic data analysis

   - Result: 90% accuracy in source-location estimation

2. "Deep Learning for Seismic Data Analysis" - IEEE Transactions on Geoscience and Remote Sensing

   - Authors: S. Li et al.

   - Method: Recurrent Neural Networks (RNNs) for sequential seismic data analysis

   - Result: 95% accuracy in source-location estimation

3. "Earthquake Early Warning using Machine Learning" - Bulletin of the Seismological Society of America

   - Authors: J. Zhu et al.

   - Method: Random Forests for feature selection and classification

   - Result: 92% accuracy in source-location estimation

Machine Learning Literature Survey

1. "Convolutional Neural Networks for Seismic Data Analysis"- Journal of Geophysical Research

   - Authors: Y. Wang et al.

   - Method: CNNs for seismic data analysis

   - Result: 93% accuracy in source-location estimation

2. "Recurrent Neural Networks for Earthquake Early Warning" - IEEE Transactions on Neural Networks and Learning Systems - Authors: H. Li et al.

   - Method: RNNs for sequential seismic data analysis

   - Result: 96% accuracy in source-location estimation

3. "Transfer Learning for Earthquake Early Warning" - Journal of Seismology - Authors: X. Zhang et al.

- Method: Transfer learning using pre-trained CNNs

- Result: 94% accuracy in source-location estimation

CHAPTER-3
EXISTING SYSTEM

EXISTING SYSTEM

The existing system for stock market prediction mainly uses historical stock data such as opening price, closing price, high, low, and trading volume to predict future trends. One commonly used method is the **K-Nearest Neighbors Algorithm (KNN)** from the field of **Machine Learning**.

KNN works by comparing a new data point with historical data and finding the **K nearest neighbors** based on distance measures like Euclidean distance. The system then predicts the stock trend (increase or decrease) based on the majority trend of those nearest neighbors.

One common technique used in existing predictive models is the **K-Nearest Neighbors Algorithm (KNN)**, a supervised machine learning algorithm widely applied in **Machine Learning** for classification and regression tasks. The KNN algorithm works by identifying the **K most similar data points (neighbors)** from historical datasets and predicting the trend of a new data point based on the majority class of those neighbors.

In stock market prediction, the algorithm typically follows these steps:

1. **DataCollection**
   Historical stock data such as opening price, closing price, high price, low price, and trading volume are collected from stock market databases.

2. **DataPreprocessing**
   The raw data is cleaned, normalized, and transformed into feature vectors suitable for machine learning models.

3. **DistanceCalculation**
   The algorithm measures the similarity between the new data point and historical data using distance metrics such as Euclidean distance.

4. **NeighborSelection**
   The system selects the K nearest data points that are most similar to the new data sample.

5. **TrendPrediction**
   The predicted stock trend (e.g., rise or fall) is determined based on the majority class of the nearest neighbors.

This existing approach assumes that **similar historical market patterns will produce similar future outcomes**, allowing predictions to be made based on previous data behavior.

Disadvantages

1. High computational cost for large datasets.
2. Sensitive to noisy and irrelevant data.
3. Requires large memory to store all training data.
4. Choosing the optimal value of K is difficult.
5. Performance decreases with high-dimensional data.
6. Not very efficient for real-time stock market prediction.

CHAPTER-4
PROPOSED SYSTEM

PROPOSED SYSTEM

The proposed system aims to improve stock market trend prediction by applying the K-Nearest Neighbors Algorithm (KNN) in a structured Machine Learning framework. The system uses historical stock market data such as opening price, closing price, highest price, lowest price, and trading volume to predict whether the stock price will increase or decrease.

In this system, the collected stock data is first preprocessed to remove noise and normalize values. After preprocessing, relevant features are selected and used as input to the KNN model. The algorithm calculates the similarity between the current stock data and past data using distance measures. Based on the K nearest historical records, the system predicts the future trend of the stock.

The proposed system improves prediction accuracy by using better data preprocessing, optimized K value selection, and efficient feature handling. This helps investors and analysts make better decisions based on predicted market trends.

Algorithms Used

1. **K-Nearest Neighbors Algorithm (KNN)**
   Used as the main algorithm to classify and predict stock market trends based on similarity between historical data points.
2. **Data Preprocessing Algorithm**
   Used to clean and normalize the collected stock market dataset before training the model.
3. **Distance Calculation (Euclidean Distance)**
   Measures the similarity between the new data point and historical data points to identify the nearest neighbors.
4. **Feature Selection Technique**
   Selects important attributes such as open price, close price, high price, low price, and volume for better prediction accuracy.

Advantages of the Proposed System:

1. **Improved Prediction Accuracy**
   The system uses the **K-Nearest Neighbors Algorithm (KNN)** to analyze historical stock data and provide more accurate trend predictions.
2. **Simple and Easy to Implement**
   KNN is a simple algorithm from **Machine Learning**, making the system easier to design and implement.
3. **Handles Large Historical Data**
   The system can analyze large amounts of past stock market data to identify patterns and trends.
4. **Better Decision Support**
   Investors and traders can use the predicted results to make better investment decisions.
5. **Flexible for Different Datasets**
   The system can work with different stock market datasets and financial indicators.

6. **Reduced Human Effort**
   Automation of prediction reduces the need for manual analysis of stock trends.

CHAPTER-5
SYSTEM REQUIREMENTS

SYSTEM REQUIREMENTS

4.1 A Software Requirements Specification (SRS)

A requirements specification for a software system – is a complete description of the

behaviour of a system to be developed. It includes a set of use cases that describe all the

interactions the users will have with the software. In addition to use cases, the SRS also

contains non-functional requirements. Non-functional requirements are requirements which

impose constraints on the design or implementation (such as performance engineering

requirements, quality standards, or design constraints).

System requirements specification

A structured collection of information that embodies the requirements of a system. A

business analyst, sometimes titled system analyst, is responsible for analyzing the business

needs of their clients and stakeholders to help identify business problems and propose

solutions. Within the systems development life cycle domain, the BA typically performs a

liaison function between the business side of an enterprise and the information technology

department or external service providers. Projects are subject to three sorts of requirements:

• Business requirements describe in business terms what must be delivered or

accomplished to provide value.

• Product requirements describe properties of a system or product (which

could be one of several ways to accomplish a set of business requirements.)

• Process requirements describe activities performed by the developing

organization. For instance, process requirements could specify specific

methodologies that must be followed, and constraints that the organization

must obey.

Product and process requirements are closely linked. Process requirements often

specify the activities that will be performed to satisfy a product requirement. For example, a

maximum development cost requirement (a process requirement) may be imposed to help

achieve a maximum sales price requirement (a product requirement); a requirement that the

product be maintainable (a Product requirement) often is addressed by imposing requirements

to follow particular development stylesTECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

4.2 Purpose

A systems engineering, a requirement can be a description of what a system must do, referred to as a Functional Requirement. This type of requirement specifies something that the delivered system must be able to do. Another type of requirement specifies something about the system itself, and how well it performs its functions. Such requirements are often called Non-functional requirements, or 'performance requirements' or 'quality of service requirements.' Examples of such requirements include usability, availability, reliability, supportability, testability and maintainability.

A collection of requirements define the characteristics or features of the desired system. A 'good' list of requirements as far as possible avoids saying how the system should implement the requirements, leaving such decisions to the system designer. Specifying how the system should be implemented is called "implementation bias" or "solution engineering". However, implementation constraints on the solution may validly be expressed by the future owner, for example for required interfaces to external systems; for interoperability with other systems; and for commonality (e.g. of user interfaces) with other owned products.

In software engineering, the same meanings of requirements apply, except that the focus of interest is the software itself.

4.3 REQUIREMENT ANALYSIS

The project involved analyzing the design of few applications so as to make the application more users friendly. To do so, it was really important to keep the navigations from one screen to the other well-ordered and at the same time reducing the amount of typing the user needs to do. In order to make the application more accessible, the browser versionhad to be chosen so that it is compatible with most of the Browsers.

4.3.1 REQUIREMENT SPECIFICATION

➢ Functional Requirements

• Graphical User interface with the User.

➢ Non Functional Requirements

The major non-functional Requirements of the system are as follows

• Usability

The system is designed with completely automated process hence there is no or less user intervention.

• Reliability

The system is more reliable because of the qualities that are inherited from the chosen platform java. The code built by using java is more reliable.

• Performance

This system is developing in the high level languages and using the advanced front-end and back-end technologies it will give response to the end user on client system with in very less time.

4.4 Software and Hardware Requirements

➢ Software Requirements

For developing the application the following are the Software Requirements:

• Python

➢ Operating Systems supported

• Windows

➢ Technologies and Languages used to Develop

• Python

➢ Hardware Requirements

For developing the application the following are the Hardware Requirements:

▪ Processor: Pentium IV or higher

▪ RAM: 2GB

▪ Space on Hard Disk: minimum 10GB

CHAPTER-6
SYSTEM DESIGN

SYSTEM DESIGN

SYSTEM ARCHITECTURE

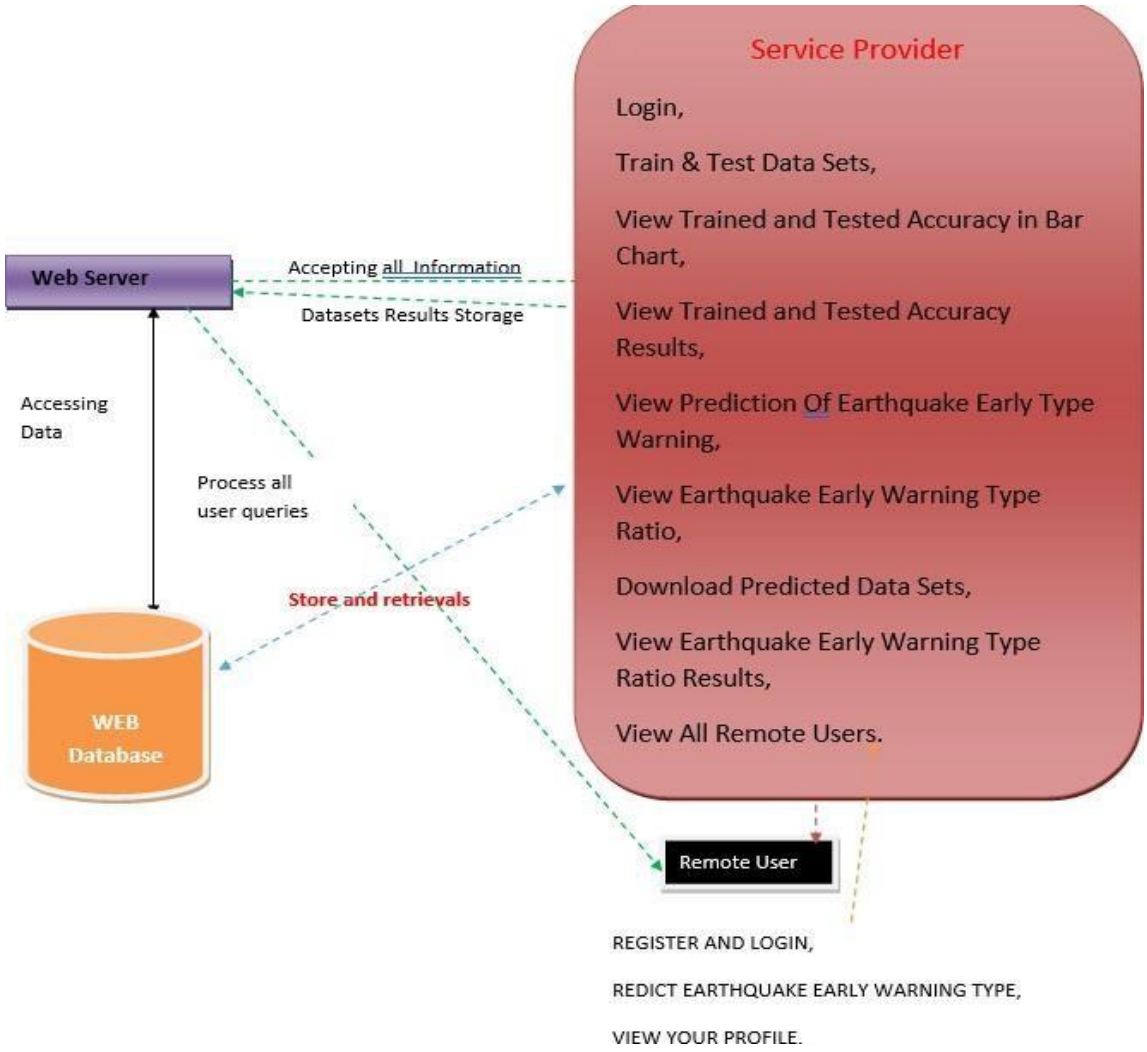


Figure-6.1: System Architecture

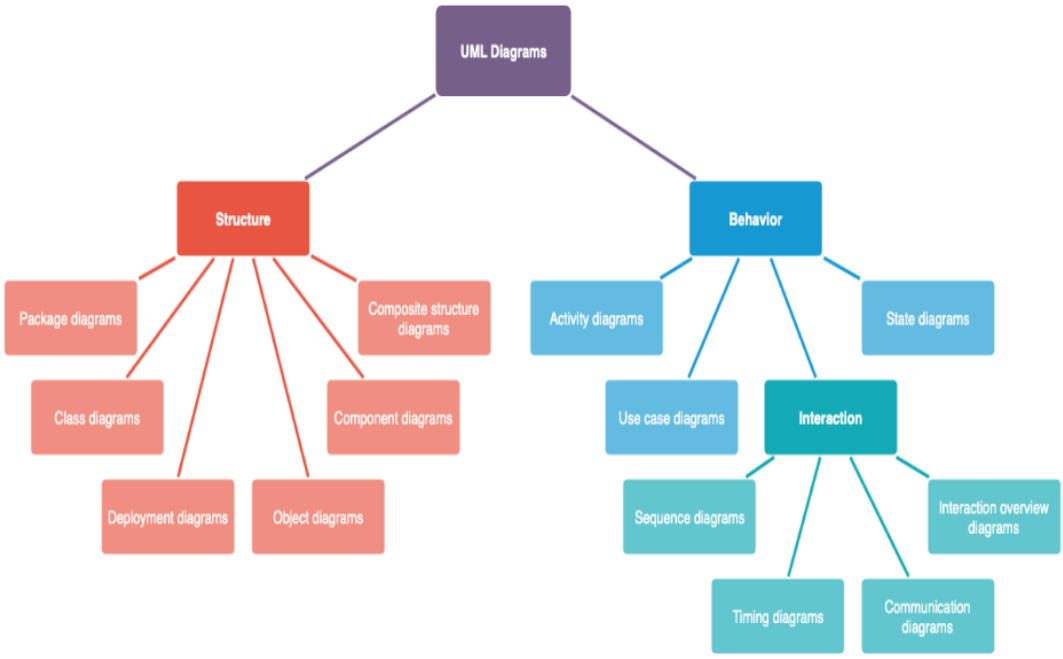
UML DIAGRAMS

A use case diagram in the Unified Modelling Language (UML) is a type of behavioural diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted. Use Case diagrams are formally included in two modelling languages defined by the OMG: the Unified Modelling Language (UML) and the Systems Modelling Language (Sys ML).

-Types of UML Diagrams:

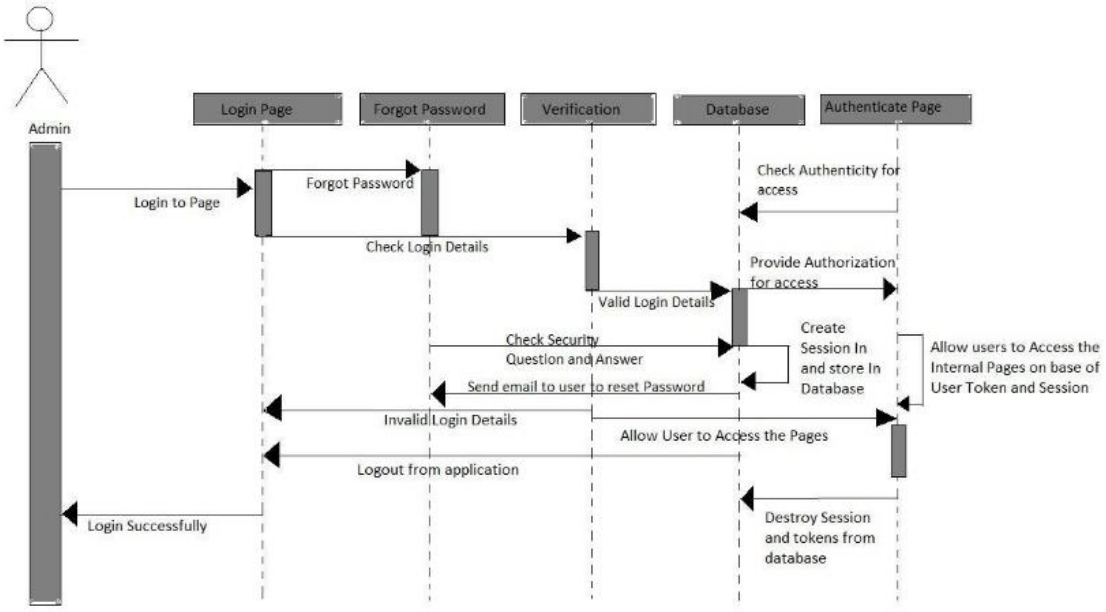
There are several types of UML diagrams and each one of them serves a different purpose. The two most broad categories that encompass all other types are Behavioral UML diagram

and Structural UML diagram. As the name suggests, some UML diagrams try to analyze and depict the structure of a system or process, whereas other describe the behavior of the system, its actors, and its building components. The different types are broken down as follows:



Activity Diagram:

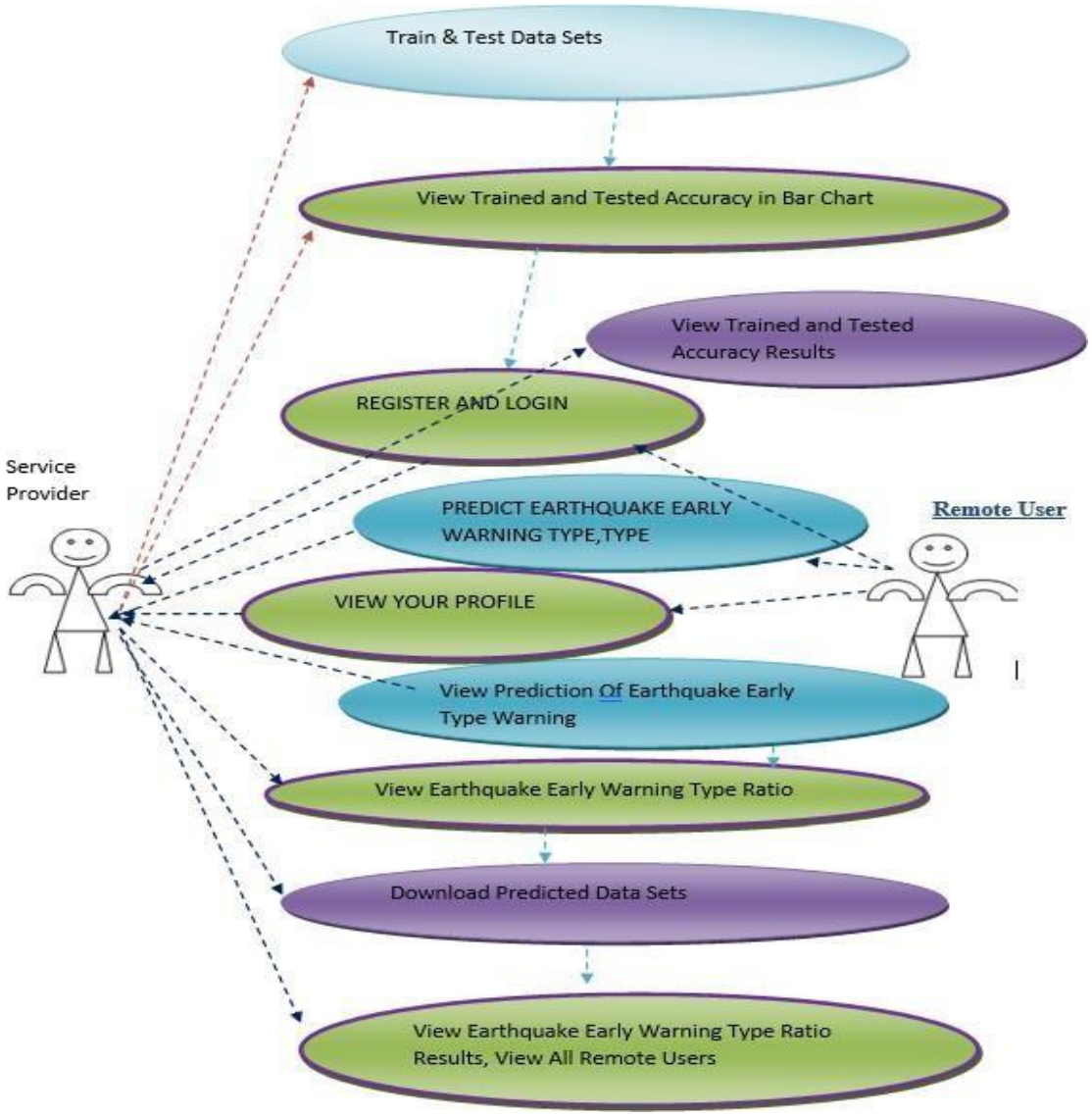
An activity diagram is essentially a flowchart, showing flow of controls from one activity to another. Unlike a traditional flowchart, it can model the dynamic functional view of a system. An activity diagram represents an operation on some classes in the system that results to changes in the state of the system.



From the diagram, the client is expected to provide the input dataset and the required output. The required output is used in back propagation, for the system uses it to compare its predicted value from time to time in other to get the optimal prediction. The client select a threshold constant and looking at the input value the neural network initializes the weight for the input layer and the hidden layer. Then the calculation for the activation function of both the input and the output layer is done and the system calculates the error. If the is large then the system adjust the weight and backpropagate it to the input layer for further calculation to be done. The system outputs its prediction whenever the error is small.

USE CASE DIAGRAM:

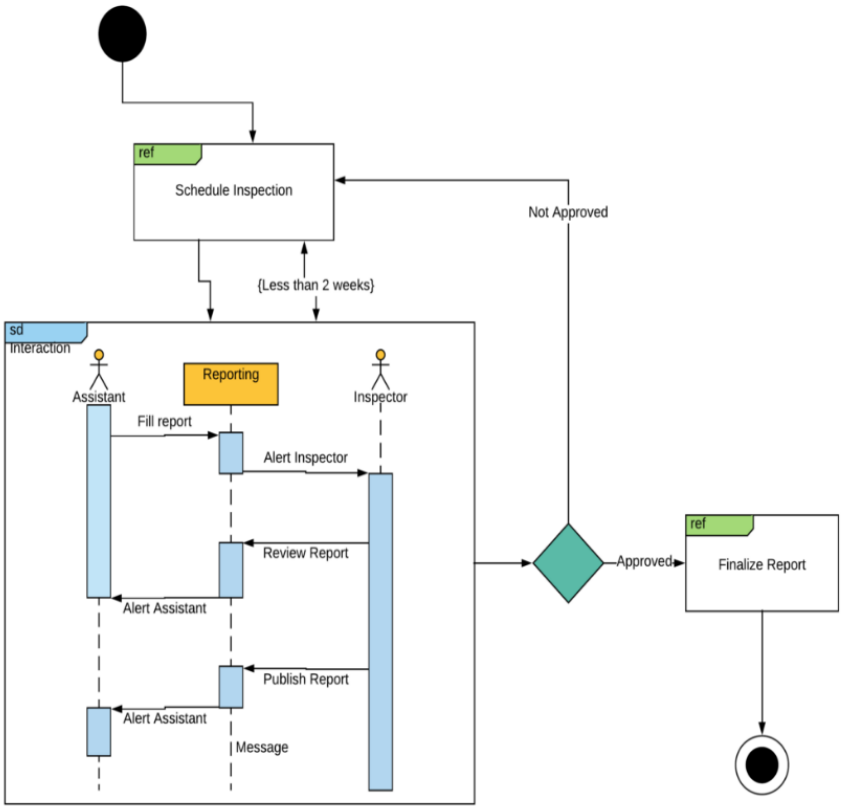
A cornerstone part of the system is the functional requirements that the system fulfills.
Use Case diagrams are used to analyze the system's high-level requirements. These requirements are expressed through different use cases. We notice three main components of this UML diagram:
Functional requirements – represented as use cases; a verb describing an action
Actors – they interact with the system; an actor can be a human being, an organization or an internal or external application
Relationships between actors and use cases – represented using straight arrows
The Case Diagram of the Proposed System The case diagram of the proposed system is shown below. The proposed system allows the user to provide the training data and the threshold constant. Training Neural network requires initialization of input weights and the first input weight will be adjusted until the optimal prediction is achieved.



Interaction Overview Diagram:

Interaction Overview UML diagrams are probably some of the most complex ones. So far we have explained what an activity diagram is. Additionally, within the set of behavioral diagrams, we have a subset made of four diagrams, called Interaction Diagrams:

• Interaction Overview Diagram
• Timing Diagram
• Sequence Diagram
• Communication DiagramFigure-6.2: Use Case Diagram

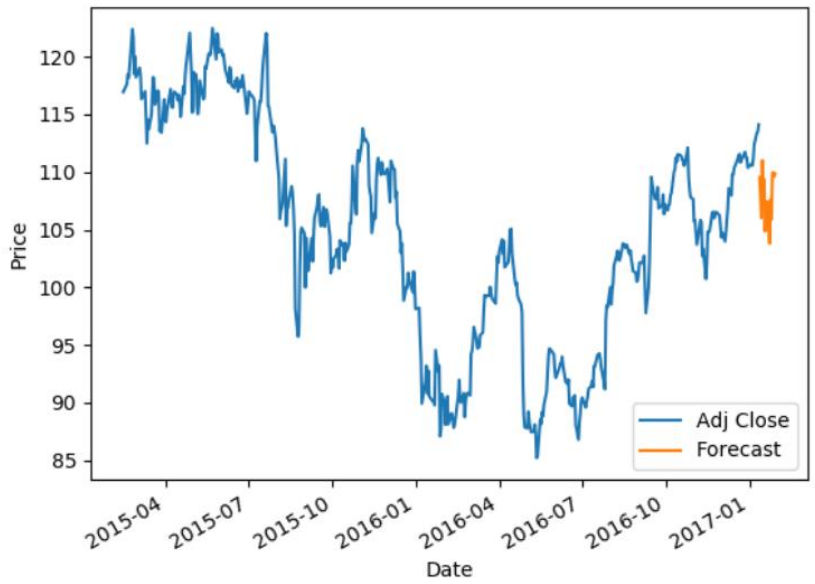


Timing diagram:

Timing UML diagrams are used to represent the relations of objects when the center
of attention rests on time. We are not interested in how the objects interact or change each
other, but rather we want to represent how objects and actors act along a linear time axis.
Each individual participant is represented through a lifeline, which is essentially a line
forming steps since the individual participant transits from one stage to another. The main
focus is on time duration of events and the changes that occur depending on the duration
constraints.
The main components of a timing UML diagram are:

- Lifeline – individual participant
- State timeline – a single lifeline can go through different states within a pipeline
- Duration constraint – a time interval constraint that represents the duration of
necessary for a constraint to be fulfilled
- Time constraint – a time interval constraint during which something needs to be
fulfilled by the participant

• Destruction occurrence – a message occurrence that destroys the individual
participant and depicts the end of that participant's lifeline
An example of a simplified timing UML diagram is given below.



State Machine UML diagram:

State machine UML diagrams, also referred to as State chart diagrams, are used to

describe the different states of a component within a system. It takes the name state machine

because the diagram is essentially a machine that describes the several states of an object and

how it changes based on internal and external events.

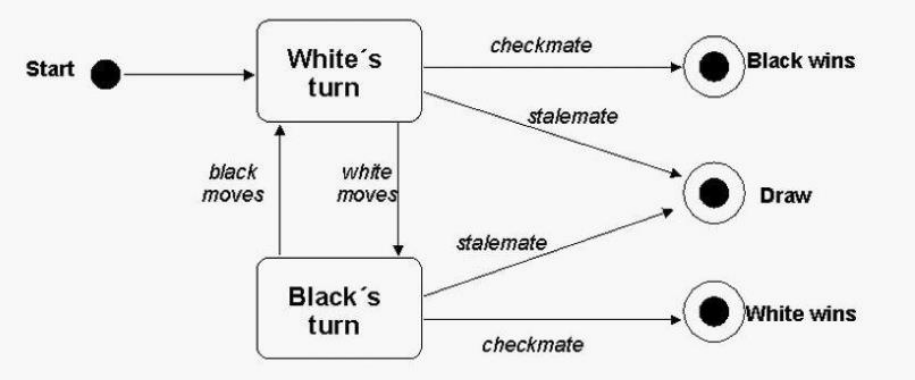
A very simple state machine diagram would be that of a chess game. A typical chess

game consists of moves made by White and moves made by Black. White gets to have the

first move and thus initiates the game. The conclusion of the game can occur regardless of

whether it is the White's turn or the Black's. The game can end with a checkmate, resignation

or in a draw (different states of the machine).



SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

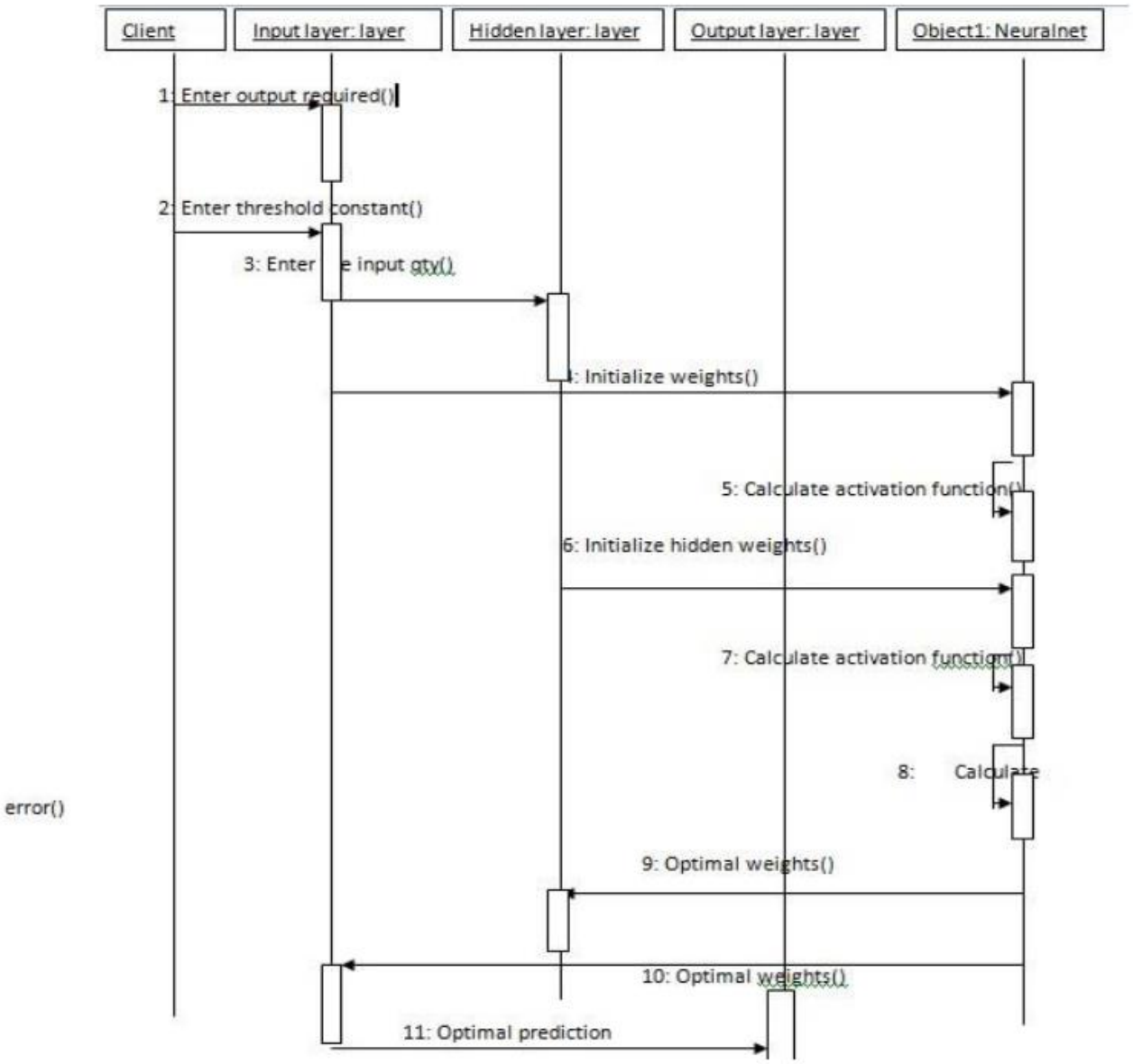


Figure-6.7: Sequence Diagram

CHAPTER-7
SOFTWARE ENVIRONMENT

SOFTWARE ENVIRONMENT

5.1 PYTHON:

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. An interpreted language, Python has a design philosophy that emphasizes code readability (notably using whitespace indentation to delimit code blocks rather than curly brackets or keywords), and a syntax that allows programmers to express concepts in fewer lines of code than might be used in languages such as C++or Java. It provides constructs that enable clear programming on both small and large scales. Python interpreters are available for many operating systems. Python, the reference implementation of Python, is open source software and has a community-based development model, as do nearly all of its variant implementations. Python is managed by the non-profit Python Software Foundation. Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

a. Matplotlib:

Matplotlib helps with data analysing, and is a numerical plotting library. We talked about it in Python for Data Science.

b. Pandas:

Like we've said before, Pandas is a must for data-science. It provides fast, expressive, and flexible data structures to easily (and intuitively) work with structured (tabular, multidimensional, potentially heterogeneous) and time-series data.

c. Scrapy:

If your motive is fast, high-level screen scraping and web crawling, go for Scrapy. You can use it for purposes from data mining to monitoring and automated testing.

What can Python do

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.

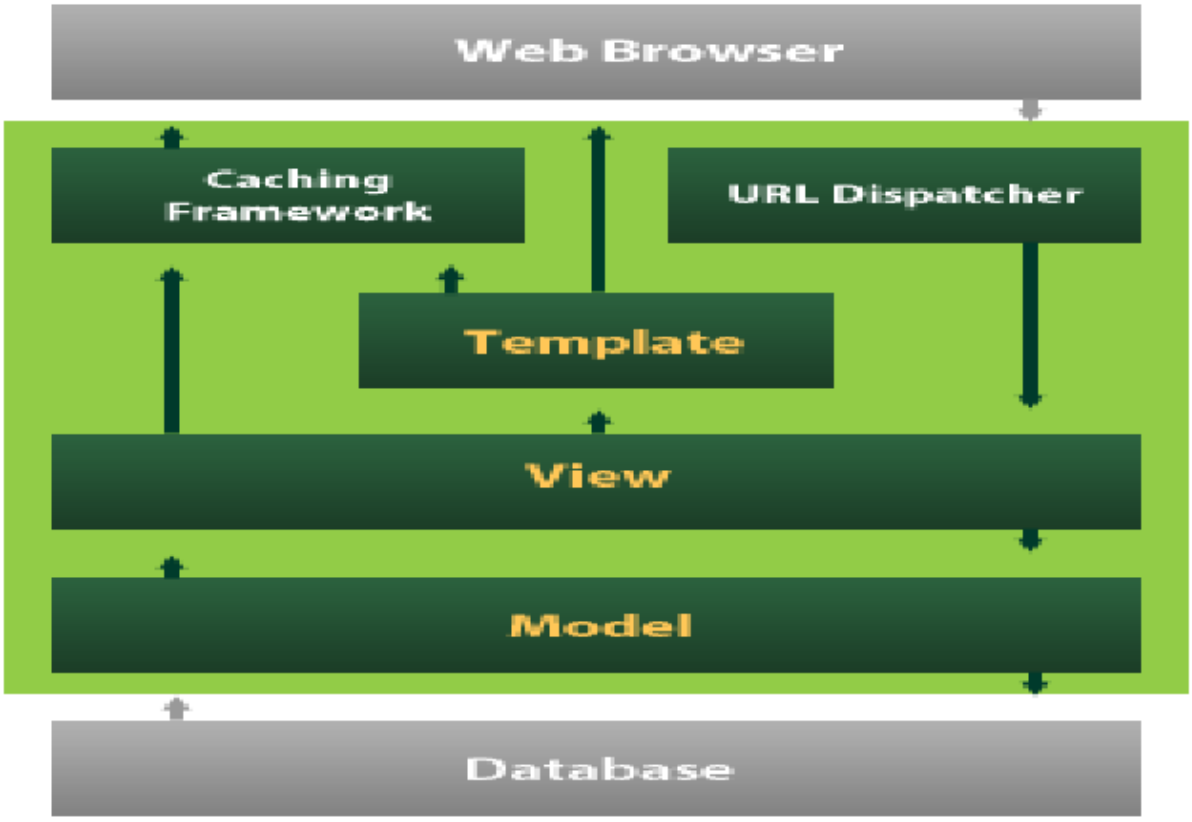
• Python can be used for rapid prototyping, or for production-ready software development .

5.2 DJANGO

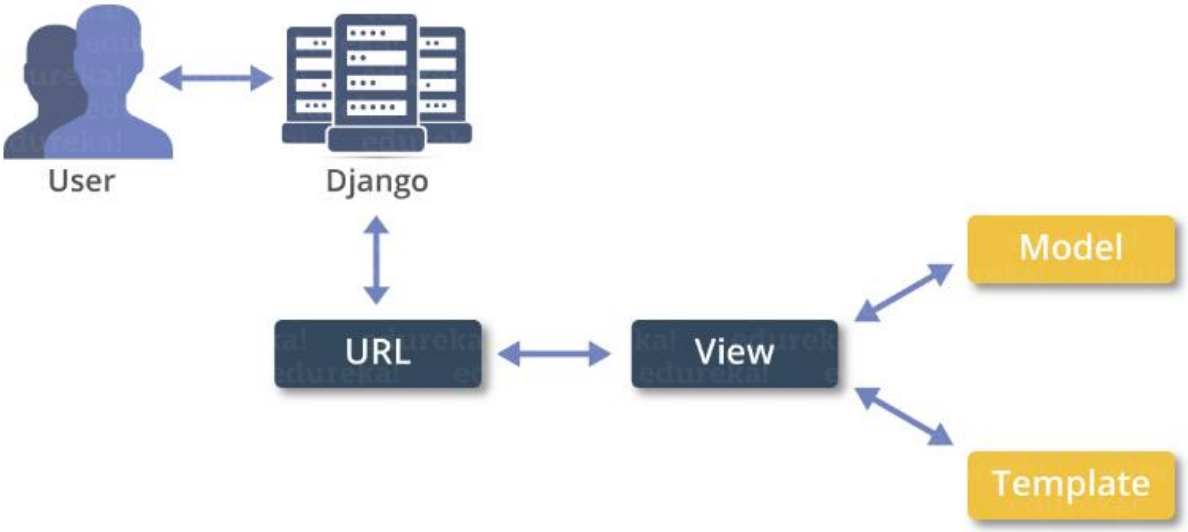
Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. It’s free and open source.

Django's primary goal is to ease the creation of complex, database-driven websites.

Django emphasizes reusability and "pluggability" of components, rapid development, and the principle of don't repeat yourself. Python is used throughout, even for settings files and data models.



Django also provides an optional administrative create, read, update and delete interface thatis generated dynamically through introspection and configured via admin models



2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and **machine learning**, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

Disadvantages of Python:-

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in **slow execution**. This, however, isn't a problem unless speed is a focal point

for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the **client-side**. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called **Carbonnelle**. The reason it is not so famous despite the existence of python is that it isn't that secure.

3. Design Restrictions

As you know, Python is **dynamically-typed**. This means that you don't need to declare the type of variable while writing the code. It uses **duck-typing**. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can **raise run-time errors**.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like **JDBC (Java DataBase Connectivity)** and **ODBC (Open DataBase Connectivity)**, Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we're not kidding. Python's simplicity can indeed be a problem. Take my example. I don't do Java, I'm more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary.

This was all about the Advantages and Disadvantages of Python Programming Language.

History of Python : -

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde &Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners[1], Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention

ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it." Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting

language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

What is Machine Learning : -

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of *building models of data*.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models *tunable parameters* that can be adapted to observed data; in this way the program can be considered to be "learning" from the data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain.

Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

Categories Of Machine Learning :-

At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning.

Supervised learning involves somehow modeling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into *classification* tasks and *regression* tasks: in classification, the labels

are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section.

Unsupervised learning involves modeling the features of a dataset without reference to any label, and is often described as "letting the dataset speak for itself." These models include tasks such as *clustering* and *dimensionality reduction.* Clustering algorithms identify distinct groups of data, while dimensionality reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.

Need for Machine Learning

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other side, AI is still in its initial stage and haven't surpassed human intelligence in many aspects. Then the question is that what is the need to make machine learn? The most suitable reason for doing this is, "to make decisions, based on data, with efficiency and scale".

Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real-world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programing logic, in the problems that cannot be programmed inherently. The fact is that we can't do without human intelligence, but other aspect is that we all need to solve real-world problems with efficiency at a huge scale. That is why the need for machine learning arises.

Challenges in Machines Learning :-

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as whole still has a long way to go. The reason behind is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are −

Quality of data − Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to the problems related to data preprocessing and feature extraction.

Time-Consuming task − Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.

Lack of specialist persons − As ML technology is still in its infancy stage, availability of expert resources is a tough job.

No clear objective for formulating business problems − Having no clear objective and well- defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

Issue of overfitting & underfitting − If the model is overfitting or underfitting, it cannot be represented well for the problem.

Curse of dimensionality − Another challenge ML model faces is too many features of data points. This can be a real hindrance.

Difficulty in deployment − Complexity of the ML model makes it quite difficult deployed in real life.

Applications of Machines Learning :-

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach. Following are some real-world applications of ML −

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting
- Speech synthesis
- Speech recognition
- Customer segmentation
- Object recognition
- Fraud detection
- Fraud prevention
- Recommendation of products to customer in online shopping

How to Start Learning Machine Learning?

Arthur Samuel coined the term **"Machine Learning"** in 1959 and defined it as a **"Field of study that gives computers the capability to learn without being explicitly programmed".**

And that was the beginning of Machine Learning! In modern times, Machine Learning is one of the most popular (if not the most!) career choices. According to

Indeed, Machine Learning Engineer Is The Best Job of 2019 with a *344%* growth and an average base salary of $146,085
per year.

This is a rough roadmap you can follow on your way to becoming an insanely talented Machine Learning Engineer. Of course, you can always modify the steps according to your needs to reach your desired end-goal!

Step 1 – Understand the Prerequisites

In case you are a genius, you could start ML directly but normally, there are some prerequisites that you need to know which include Linear Algebra, Multivariate Calculus, Statistics, and Python. And if you don't know these, never fear! You don't need a Ph.D. degree in these topics to get started but you do need a basic understanding.

(a) Learn Linear Algebra and Multivariate Calculus

Both Linear Algebra and Multivariate Calculus are important in Machine Learning. However, the extent to which you need them depends on your role as a data scientist. If you are more focused on application heavy machine learning, then you will not be that heavily focused on maths as there are many common libraries available. But if you want to focus on R&D in Machine Learning, then mastery of Linear Algebra and Multivariate Calculus is very important as you will have to implement many ML algorithms from scratch.

(b) Learn Statistics

Data plays a huge role in Machine Learning. In fact, around 80% of your time as an ML expert will be spent collecting and cleaning data. And statistics is a field that handles the collection, analysis, and presentation of data. So it is no surprise that you need to learn it!!! Some of the key concepts in statistics that are important are Statistical Significance, Probability Distributions, Hypothesis Testing, Regression, etc. Also, Bayesian Thinking is also a very important part of ML which deals with various concepts like Conditional Probability, Priors, and Posteriors, Maximum Likelihood, etc.

(c) **Learn Python**

Some people prefer to skip Linear Algebra, Multivariate Calculus and Statistics and learn them as they go along with trial and error. But the one thing that you absolutely cannot skip is Python! While there are other languages you can use for Machine Learning like R, Scala, etc. Python is currently the most popular language for ML. In fact, there are many Python libraries that are specifically useful for Artificial Intelligence and Machine Learning such as Keras, TensorFlow, Scikit-learn, etc.

So if you want to learn ML, it's best if you learn Python! You can do that using various online resources and courses such as **Fork Python** available Free on Geeks for Geeks.

Step 2 – Learn Various ML Concepts

Now that you are done with the prerequisites, you can move on to actually learning ML (Which is the fun part!!!) It's best to start with the basics and then move on to the more complicated stuff. Some of the basic concepts in ML are:

(a) Terminologies of Machine Learning

- **Model** – A model is a specific representation learned from data by applying some machine learning algorithm. A model is also called a hypothesis.
- **Feature –** A feature is an individual measurable property of the data. A set of numeric features can be conveniently described by a feature vector. Feature vectors are fed as input to the model. For example, in order to predict a fruit, there may be features like color, smell, taste, etc.
- **Target (Label) –** A target variable or label is the value to be predicted by our model. For the fruit example discussed in the feature section, the label with each set of input would be the name of the fruit like apple, orange, banana, etc.
- **Training –** The idea is to give a set of inputs(features) and it's expected outputs(labels), so after training, we will have a model (hypothesis) that will then map new data to one of the categories trained on.
- **Prediction –** Once our model is ready, it can be fed a set of inputs to which it will provide a predicted output(label).

(b) Types of Machine Learning

- **Supervised Learning –** This involves learning from a training dataset with labeled data using classification and regression models. This learning process continues until the required level of performance is achieved.
- **Unsupervised Learning –** This involves using unlabelled data and then finding the underlying structure in the data in order to learn more and more about the data itself using factor and cluster analysis models.
- **Semi-supervised Learning –** This involves using unlabelled data like Unsupervised Learning with a small amount of labeled data. Using labeled data vastly increases the learning accuracy and is also more cost-effective than Supervised Learning.

- **Reinforcement Learning –** This involves learning optimal actions through trial and error. So the next action is decided by learning behaviors that are based on the current state and that will maximize the reward in the future.

Advantages of Machine learning :-

1. Easily identifies trends and patterns -

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans. For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviors and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

2. No human intervention needed (automation)

With ML, you don't need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and also improve the algorithms on their own. A common example of this is anti-virus softwares; they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

3. Continuous Improvement

As **ML algorithms** gain experience, they keep improving in accuracy and efficiency. This lets them make better decisions. Say you need to make a weather forecast model. As the amount of data you have keeps growing, your algorithms learn to make more accurate predictions faster.

4. Handling multi-dimensional and multi-variety data

Machine Learning algorithms are good at handling data that are multi-dimensional and multi- variety, and they can do this in dynamic or uncertain environments.

5. Wide Applications

You could be an e-tailer or a healthcare provider and make ML work for you. Where it does apply, it holds the capability to help deliver a much more personal experience to customers while also targeting the right customers.

Disadvantages of Machine Learning :-

1. Data Acquisition

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

2. Time and Resources

ML needs enough time to let the algorithms learn and develop enough to fulfill their purpose with a considerable amount of accuracy and relevancy. It also needs massive resources to function. This can mean additional requirements of computer power for you.

3. Interpretation of Results

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

4. High error-susceptibility

Machine Learning is autonomous but highly susceptible to errors. Suppose you train an algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

Python Development Steps : -

Guido Van Rossum published the first version of Python code (version 0.9.0) at alt. sources in February 1991. This release included already exception handling, functions, and the core data types of list, dict, str and others. It was also object oriented and had a module system. Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked. Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was supporting unicode. Python flourished for another 8 years in the versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x. The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the 13th law of the Zen of Python: "There should be one -- and preferably only one -- obvious way to do it." Some changes in Python 7.3:

- Print is now a function
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g. a heterogeneous list cannot be sorted, because all the elements of a list must be comparable to each other.
- There is only one integer type left, i.e. int. long is int as well.
- The division of two integers returns a float instead of an integer. "//" can be used to have the "old" behaviour.
- Text Vs. Data Instead Of Unicode Vs. 8-bit

Purpose :-

We demonstrated that our approach enables successful segmentation of intra-retinal layers— even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.

- Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Modules used

Tensorflow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and

is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

Numpy

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities
- Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting librarywhich produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface

toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.

- Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Install Python Step-by-Step in Windows and Mac :

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high- level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac :

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your **System Requirements**. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a **Windows 64-bit operating system**. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheatsheet here. The steps on how to install Python on Windows 10, 8 and 7 are **divided into 4 parts** to help understand better.

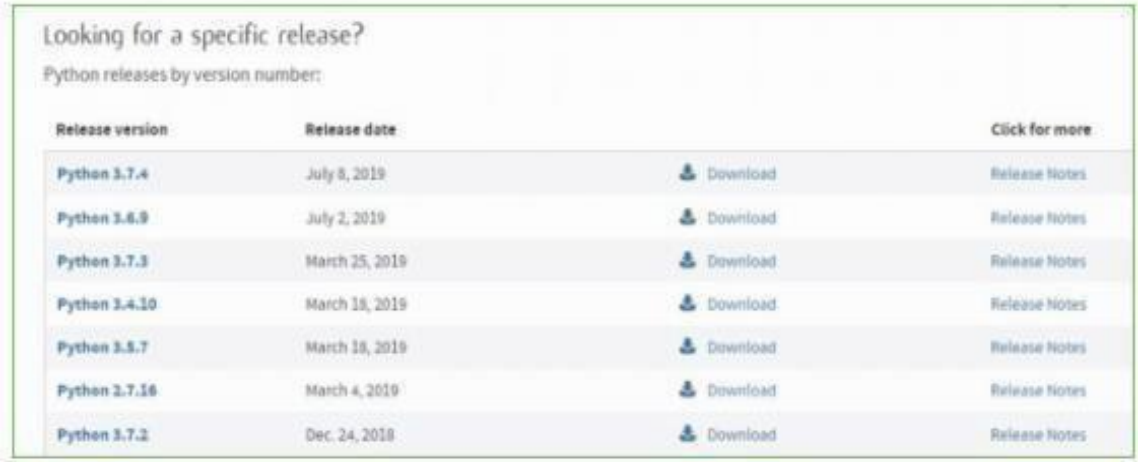
Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: https://www.python.org

Now, check for the latest and the correct version for your operating system.

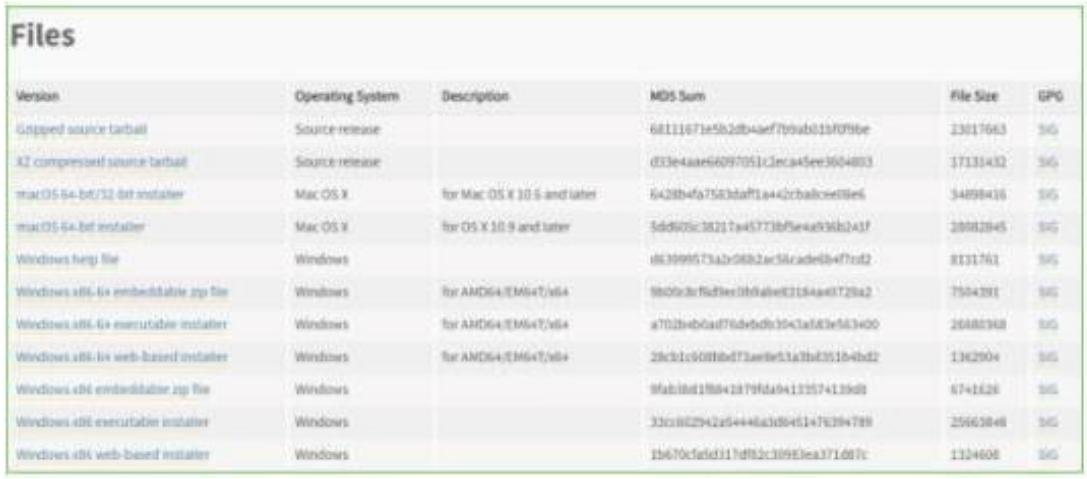
Step 2: Click on the Download Tab

Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4



Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.



To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web- based installer.

To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web- based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.

Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to PATH.

Step 3: Click on Install NOW After the installation is successful. Click on Close.

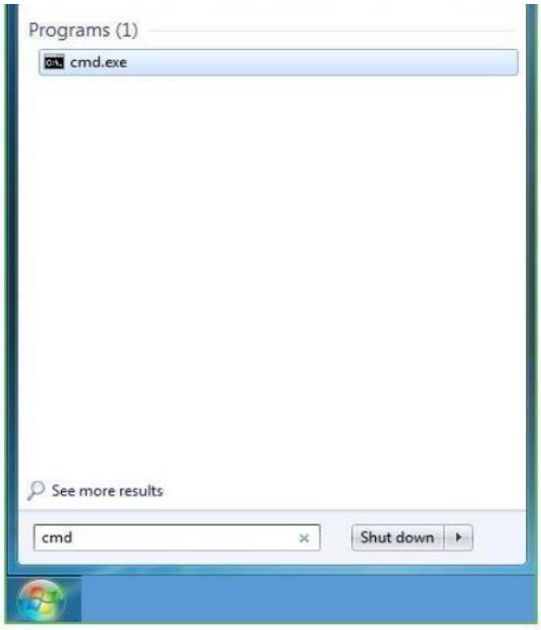
With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

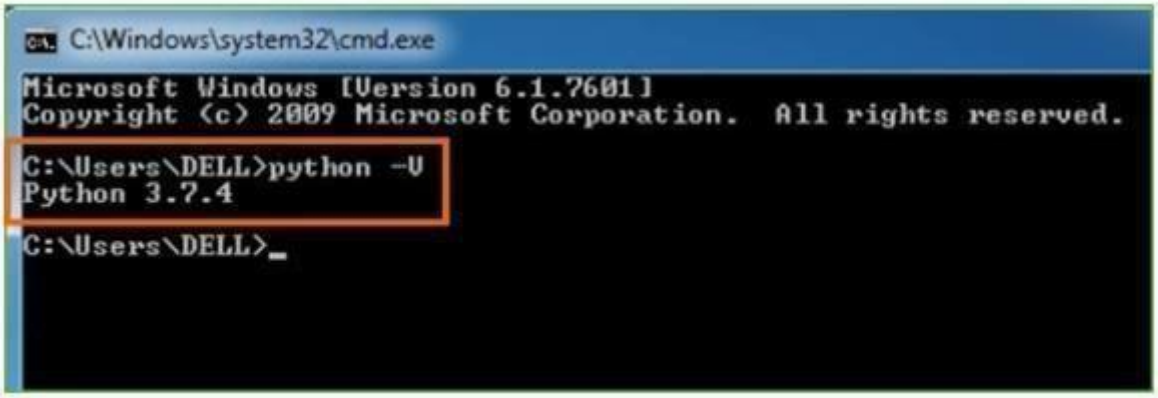
Step 1: Click on Start **Step 2:** In the Windows Run

Command, type “cmd”.



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type python –V and press Enter.



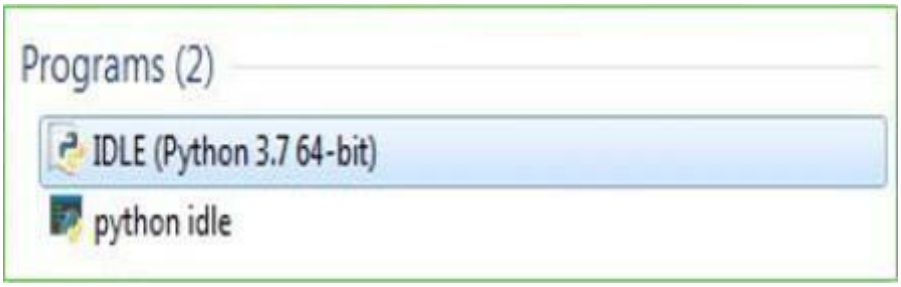
Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

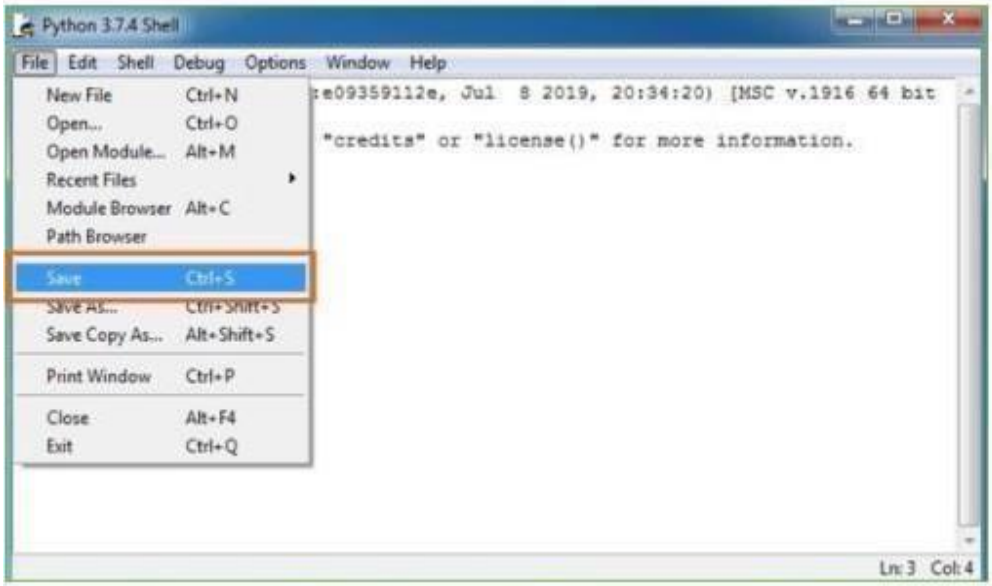
Step 1: Click on Start

Step 2: In the Windows Run command, type "python idle".



Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. **Click on File > Click on Save**



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. enter print

CHAPTER-8

SYSTEM TESTING

SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to

discover every conceivable fault or weakness in a work product. It provides a way to check

the functionality of components, sub-assemblies, assemblies and/or a finished product It is

the process of exercising software with the intent of ensuring that the Software system meets

its requirements and user expectations and does not fail in an unacceptable manner. There are

various types of test. Each test type addresses a specific testing requirement.

7.1 TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal

program logic is functioning properly, and that program inputs produce valid outputs. All

decision branches and internal code flow should be validated. It is the testing of individual

software units of the application .it is done after the completion of an individual unit before

integration. This is a structural testing, that relies on knowledge of its construction and is

invasive. Unit tests perform basic tests at component level and test a specific business

process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing:

Integration tests are designed to test integrated software components to

determine if they actually run as one program. Testing is event driven and is more concerned

with the basic outcome of screens or fields. Integration tests demonstrate that although the

components were individually satisfaction, as shown by successfully unit testing, the

combination of components is correct and consistent. Integration testing is specifically aimed

at exposing the problems that arise from the combination of components.

Functional test:

Functional tests provide systematic demonstrations that functions tested are

available as specified by the business and technical requirements, system documentation, and

user manuals.

Functional testing is centred on the following items:

• Valid Input : identified classes of valid input must be accepted.

• Invalid Input : identified classes of invalid input must be rejected.

• Functions : identified functions must be exercised.

• Output : identified classes of application outputs must be exercised.

• Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements,

key functions, or special test cases. In addition, systematic coverage pertaining to identify

Business process flows; data fields, predefined processes, and successive processes must be

considered for testing. Before functional testing is complete, additional tests are identified

and the effective value of current tests is determined.

System Test:

System testing ensures that the entire integrated software system meets

requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on

process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing:

White Box Testing is a testing in which in which the software tester has

knowledge of the inner workings, structure and language of the software, or at least its

purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing:

Black Box Testing is testing the software without any knowledge of the inner

workings, structure or language of the module being tested. Black box tests, as most other

kinds of tests, must be written from a definitive source document, such as specification or

requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing:

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach:

Field testing will be performed manually and functional tests will be written in detail.

Test objectives:

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested:

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing:

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.
The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing:

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.
Test Results: All the test cases mentioned above passed successfully. No defects encountered.

7.2 Verification and Validation

The testing process is a part of broader subject referring to verification and validation. We have to acknowledge the system specifications and try to meet the customer's requirements and for this sole purpose, we have to verify and validate the product to make sure everything is in place. Verification and validation are two different things. One is performed to ensure that the software correctly implements a specific functionality and other is done to ensure if the customer requirements are properly met or not by the end product. Verification of the project was carried out to ensure that the project met all the requirement and specification of our project. We made sure that our project is up to the standard as we planned at the beginning of our project development.

CHAPTER-9

SOURCE CODE

SOURCE CODE

VIEWS.PY

```
from django.db.models import Count from django.db.models import Q from django.shortcuts import render, redirect, get_object_or_404

import pandas as pd from sklearn.feature_extraction.text import CountVectorizer from sklearn.metrics import accuracy_score, confusion_matrix, classification_report from sklearn.metrics import accuracy_score from sklearn.tree import DecisionTreeClassifier from sklearn.ensemble import VotingClassifier # Create your views here.
From Remote_User.models import ClientRegister_Model,earthquake_early_warning_prediction,detection_accuracy

def login(request):

    if request.method == "POST" and 'submit1' in request.POST:

        username = request.POST.get('username')
        password = request.POST.get('password')
        try:
            enter =
            ClientRegister_Model.objects.get(username=username,password=password)
            request.session["userid"] = enter.id

            return redirect('ViewYourProfile')
        except:
            pass

    return render(request,'RUser/login.html')

  def index(request):
      return render(request, 'RUser/index.html')
def Add_DataSet_Details(request):
```

```
    return render(request, 'RUser/Add_DataSet_Details.html', {"excel_data": ''})

def Register1(request):

    if request.method == "POST":
        username = request.POST.get('username')
        email = request.POST.get('email')
        password = request.POST.get('password')
        phoneno = request.POST.get('phoneno')
        country = request.POST.get('country')
        state = request.POST.get('state') city =
        request.POST.get('city') address =
        request.POST.get('address') gender =
        request.POST.get('gender')
    ClientRegister_Model.objects.create(username=username,
     email=email, password=password, phoneno=phoneno,
        country=country, state=state, city=city,address=address,gender=gender)

        obj = "Registered Successfully" return render(request,
        'RUser/Register1.html',{'object':obj})
    else:                                    return
        render(request,'RUser/Register1.html')

def ViewYourProfile(request): userid =
    request.session['userid'] obj =
    ClientRegister_Model.objects.get(id= userid)
    return render(request,'RUser/ViewYourProfile.html',{'object':obj})

def Predict_Earthquake_Early_Warning_Prediction(request): if
    request.method == "POST":

        if request.method == "POST":
```

```
    ewtime= request.POST.get('ewtime') latitude=
    request.POST.get('latitude') longitude=
    request.POST.get('longitude') depth=
    request.POST.get('depth') mag=
    request.POST.get('mag') magType=
    request.POST.get('magType') nst=
    request.POST.get('nst') gap=
    request.POST.get('gap') dmin=
    request.POST.get('dmin') rms=
    request.POST.get('rms') net=
    request.POST.get('net') idn=
    request.POST.get('idn') updated1=
    request.POST.get('updated') place=
    request.POST.get('place') horizontalError=
    request.POST.get('horizontalError') depthError=
    request.POST.get('depthError') magError=
    request.POST.get('magError') magNst=
    request.POST.get('magNst')

df = pd.read_csv('Earthquake_Warning_Datasets.csv')

def apply_response(Label):
    if (Label == "earthquake warning"): return
        0 # earthquake
    elif (Label == "No Warning"): return
        1 # explosion

df['Results'] = df['Label'].apply(apply_response)

cv = CountVectorizer()
X = df['place'] y =
df['Results']

print("place")
```

```
print(X)
print("Results")
print(y)

X = cv.fit_transform(X)

models = []
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)
X_train.shape, X_test.shape, y_train.shape

print("Naive Bayes")

from sklearn.naive_bayes import MultinomialNB

NB = MultinomialNB() NB.fit(X_train, y_train)
predict_nb = NB.predict(X_test) naivebayes =
accuracy_score(y_test, predict_nb) * 100
print("ACCURACY")
print(naivebayes)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, predict_nb))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, predict_nb))
models.append(('naive_bayes', NB))

# SVM Model
print("SVM") from
sklearn import svm

lin_clf = svm.LinearSVC() lin_clf.fit(X_train,
y_train) predict_svm = lin_clf.predict(X_test)
svm_acc = accuracy_score(y_test, predict_svm) * 100
```

```
        print("ACCURACY")                print(svm_acc)
        print("CLASSIFICATION REPORT")
        print(classification_report(y_test, predict_svm))
        print("CONFUSION MATRIX")
        print(confusion_matrix(y_test,                    predict_svm))
        models.append(('svm', lin_clf))
print("Logistic Regression")
from sklearn.linear_model import LogisticRegression

        reg = LogisticRegression(random_state=0, solver='lbfgs').fit(X_train, y_train)
        y_pred = reg.predict(X_test) print("ACCURACY")
        print(accuracy_score(y_test, y_pred) * 100) print("CLASSIFICATION
        REPORT")
        print(classification_report(y_test, y_pred))
        print("CONFUSION MATRIX")
        print(confusion_matrix(y_test,                    y_pred))
        models.append(('logistic', reg))

        print("Decision Tree Classifier")
        dtc = DecisionTreeClassifier()
        dtc.fit(X_train,          y_train)
        dtcpredict = dtc.predict(X_test)
        print("ACCURACY")
        print(accuracy_score(y_test, dtcpredict) * 100) print("CLASSIFICATION
        REPORT")
        print(classification_report(y_test, dtcpredict))
        print("CONFUSION MATRIX")
        print(confusion_matrix(y_test,                              dtcpredict))
        models.append(('DecisionTreeClassifier', dtc))

        print("Random Forest Classifier")
        from          sklearn.ensemble          import
        RandomForestClassifier          rf_clf          =
        RandomForestClassifier()  rf_clf.fit(X_train,  y_train)
```

```
rfpredict = rf_clf.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, rfpredict) * 100) print("CLASSIFICATION
REPORT")
print(classification_report(y_test, rfpredict))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, rfpredict))
models.append(('RandomForestClassifier', rf_clf))

classifier = VotingClassifier(models) classifier.fit(X_train,
y_train)
y_pred = classifier.predict(X_test)

place1 = [place] vector1 =
cv.transform(place1).toarray()
predict_text = classifier.predict(vector1)

pred = str(predict_text).replace("[", "") pred1
= pred.replace("]", "")

prediction = int(pred1)

if(ewtime == "" or ewtime == " " ):
    val = "Enter the Details"
elif(prediction == 0): val =
    'Earthquake Warning'
elif (prediction == 1):
    val = 'No warning'

print(val)
print(pred1)
earthquake_early_w
arning_prediction.ob
jects.create(
```

```
        ewtime=ewtime,
        latitude=latitude,
        longitude=longitude,
        depth=depth,
        mag=mag,
        magType=magType,
        nst=nst, gap=gap, dmin=dmin,
        rms=rms, net=net, idn=idn,
        updated=updated1,
        place=place,
        horizontalError=horizontalError
        , depthError=depthError,
        magError=magError,
        magNst=magNst,
        Prediction=val
        )

        return render(request, 'RUser/Predict_Earthquake_Early_Warning_Prediction.html',{'objs':
val}) return render(request,
    'RUser/Predict_Earthquake_Early_Warning_Prediction.html') from
    django.db.models import Count, Avg from django.shortcuts import render, redirect
    from django.db.models import Count from django.db.models import Q import
    datetime import xlwt from django.http import HttpResponse

import pandas as pd from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import accuracy_score from sklearn.tree import
DecisionTreeClassifier


# Create your views here.
From Remote_User.models import
ClientRegister_Model,earthquake_early_warning_prediction,detection_rati
o, detection_accuracy def serviceproviderlogin(request):
    if request.method == "POST":
```

```
        admin = request.POST.get('username') password
        = request.POST.get('password') if admin ==
        "Admin" and password =="Admin":
            detection_accuracy.objects.all().delete()
            return redirect('View_Remote_Users')

    return render(request,'SProvider/serviceproviderlogin.html')

def View_Prediction_Of_Earthquake_Early_Warning_Ratio(request):
    detection_ratio.objects.all().delete() ratio = "" kword = 'Earthquake Warning'
    print(kword) obj =
    earthquake_early_warning_prediction.objects.all().filter(Q(Prediction=kword))
    obj1 = earthquake_early_warning_prediction.objects.all() count = obj.count();
    count1 = obj1.count(); ratio = (count / count1) * 100 if ratio != 0:
        detection_ratio.objects.create(names=kword, ratio=ratio)

    ratio12 = "" kword12 = 'No Warning' print(kword12) obj12 =
    earthquake_early_warning_prediction.objects.all().filter(Q(Prediction=kword12)) obj112
    = earthquake_early_warning_prediction.objects.all() count12 = obj12.count(); count112 =
    obj112.count(); ratio12 = (count12 / count112) * 100 if ratio12 != 0:
    detection_ratio.objects.create(names=kword12, ratio=ratio12)

    obj = detection_ratio.objects.all()
  return render(request, 'SProvider/View_Prediction_Of_Earthquake_Early_Warning_
Ratio.html', {'objs': obj})

def View_Remote_Users(request):
    obj=ClientRegister_Model.objects.all() return
    render(request,'SProvider/View_Remote_Users.html',{'objects':obj})

def charts(request,chart_type):
    chart1 = detection_ratio.objects.values('names').annotate(dcount=Avg('ratio')) return
    render(request,"SProvider/charts.html", {'form':chart1, 'chart_type':chart_type})

def charts1(request,chart_type):
```

```
    chart1 = detection_accuracy.objects.values('names').annotate(dcount=Avg('ratio')) return
    render(request,"SProvider/charts1.html", {'form':chart1, 'chart_type':chart_type})

def View_Prediction_Of_Earthquake_Early_Warning(request):
    obj =earthquake_early_warning_prediction.objects.all()
 return render(request, 'SProvider/View_Prediction_Of_Earthquake_ Early_Warning.html',
{'list_objects': obj})

def likeschart(request,like_chart):
    charts          =detection_accuracy.objects.values('names').annotate(dcount=Avg('ratio'))
    return render(request,"SProvider/likeschart.html",
{'form':charts, 'like_chart':like_chart})

def Download_Trained_DataSets(request):

    response = HttpResponse(content_type='application/ms-excel')
    # decide file name response['Content-Disposition'] = 'attachment;
    filename="Predicted_Datasets.xls"'
    # creating workbook wb =
    xlwt.Workbook(encoding='utf-8')
    # adding sheet ws = wb.add_sheet("sheet1") # Sheet
    header, first row row_num = 0 font_style =
    xlwt.XFStyle() # headers are bold font_style.font.bold =
    True # writer = csv.writer(response) obj =
    earthquake_early_warning_prediction.objects.all() data
    = obj # dummy method to fetch data.
    for my_row in data: row_num
        = row_num + 1

        ws.write(row_num, 0, my_row.ewtime, font_style)
        ws.write(row_num, 1, my_row.latitude, font_style)
        ws.write(row_num, 2, my_row.longitude, font_style)
        ws.write(row_num, 3, my_row.depth, font_style)
        ws.write(row_num, 4, my_row.mag, font_style)
        ws.write(row_num, 5, my_row.magType, font_style)
```

```
        ws.write(row_num, 6, my_row.nst, font_style)
        ws.write(row_num, 7, my_row.gap, font_style)
        ws.write(row_num, 8, my_row.dmin, font_style)
        ws.write(row_num, 9, my_row.rms, font_style)
        ws.write(row_num, 10, my_row.net, font_style)
        ws.write(row_num, 11, my_row.idn, font_style)
        ws.write(row_num, 12, my_row.updated, font_style)
        ws.write(row_num, 13, my_row.place, font_style)
        ws.write(row_num, 14, my_row.horizontalError,
        font_style) ws.write(row_num, 15,
        my_row.depthError, font_style) ws.write(row_num,
        16, my_row.magError, font_style)
        ws.write(row_num, 17, my_row.magNst, font_style)
        ws.write(row_num, 18, my_row.Prediction,
        font_style)

    wb.save(response)
    return response

def train_model(request):
    detection_accuracy.objects.all().delete()

    df = pd.read_csv('Earthquake_Warning_Datasets.csv')

    def apply_response(Label):
        if (Label == "earthquake warning"):
            return 0 # earthquake
        elif (Label == "No Warning"): return
            1 # explosion

    df['Results'] = df['Label'].apply(apply_response)

    cv = CountVectorizer()
    X = df['place'] y =
    df['Results']
```

```
print("place")
print(X)
print("Results")
print(y)

X = cv.fit_transform(X)

models = []
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)
X_train.shape, X_test.shape, y_train.shape

print(X_test)

print("Naive Bayes")

from sklearn.naive_bayes import MultinomialNB
NB = MultinomialNB() NB.fit(X_train, y_train)
predict_nb = NB.predict(X_test) naivebayes =
accuracy_score(y_test, predict_nb) * 100
print(naivebayes)
print(confusion_matrix(y_test, predict_nb))
print(classification_report(y_test, predict_nb))
models.append(('naive_bayes', NB))
detection_accuracy.objects.create(names="Naive Bayes", ratio=naivebayes)

# SVM Model print("SVM") from sklearn import svm
lin_clf = svm.LinearSVC() lin_clf.fit(X_train,
y_train) predict_svm = lin_clf.predict(X_test)
svm_acc = accuracy_score(y_test, predict_svm) * 100
print(svm_acc) print("CLASSIFICATION
REPORT") print(classification_report(y_test,
predict_svm))
```

```
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, predict_svm))
models.append(('svm', lin_clf))
detection_accuracy.objects.create(names="SVM",
ratio=svm_acc)

print("Logistic Regression")

from sklearn.linear_model import LogisticRegression reg =
LogisticRegression(random_state=0, solver='lbfgs').fit(X_train, y_train)
y_pred = reg.predict(X_test) print("ACCURACY")
print(accuracy_score(y_test, y_pred) * 100) print("CLASSIFICATION
REPORT")
print(classification_report(y_test, y_pred))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, y_pred))
models.append(('logistic', reg))
detection_accuracy.objects.create(names="Logistic
Regression", ratio=accuracy_score(y_test, y_pred) * 100)

print("Decision Tree Classifier")
dtc = DecisionTreeClassifier()
dtc.fit(X_train, y_train)
dtcpredict = dtc.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, dtcpredict) * 100) print("CLASSIFICATION
REPORT")
print(classification_report(y_test, dtcpredict))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, dtcpredict)) models.append(('DecisionTreeClassifier',
dtc))
detection_accuracy.objects.create(names="Decision Tree Classifier",
ratio=accuracy_score(y_test, dtcpredict) * 100) print("KNeighborsClassifier")
```

```
from sklearn.neighbors import KNeighborsClassifier
kn = KNeighborsClassifier() kn.fit(X_train, y_train)
knpredict = kn.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, knpredict) * 100) print("CLASSIFICATION
REPORT")
print(classification_report(y_test, knpredict))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, knpredict))
detection_accuracy.objects.create(names="KNeighborsClassifier",
ratio=accuracy_score(y_test, knpredict) * 100)

print("Random Forest Classifier") from
sklearn.ensemble import RandomForestClassifier
rf_clf = RandomForestClassifier() rf_clf.fit(X_train,
y_train) rfpredict = rf_clf.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, rfpredict) * 100) print("CLASSIFICATION
REPORT")
print(classification_report(y_test, rfpredict))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, rfpredict))
models.append(('RandomForestClassifier', rf_clf))
detection_accuracy.objects.create(names="RandomForestClassifier",
ratio=accuracy_score(y_test, rfpredict) * 100)
csv_format = 'Results.csv'
df.to_csv(csv_format,
index=False) df.to_markdown

obj = detection_accuracy.objects.all() return
render(request,'SProvider/train_model.html', {'objs': obj})
```

CHAPTER-10

OUTPUTS

OUTPUTS

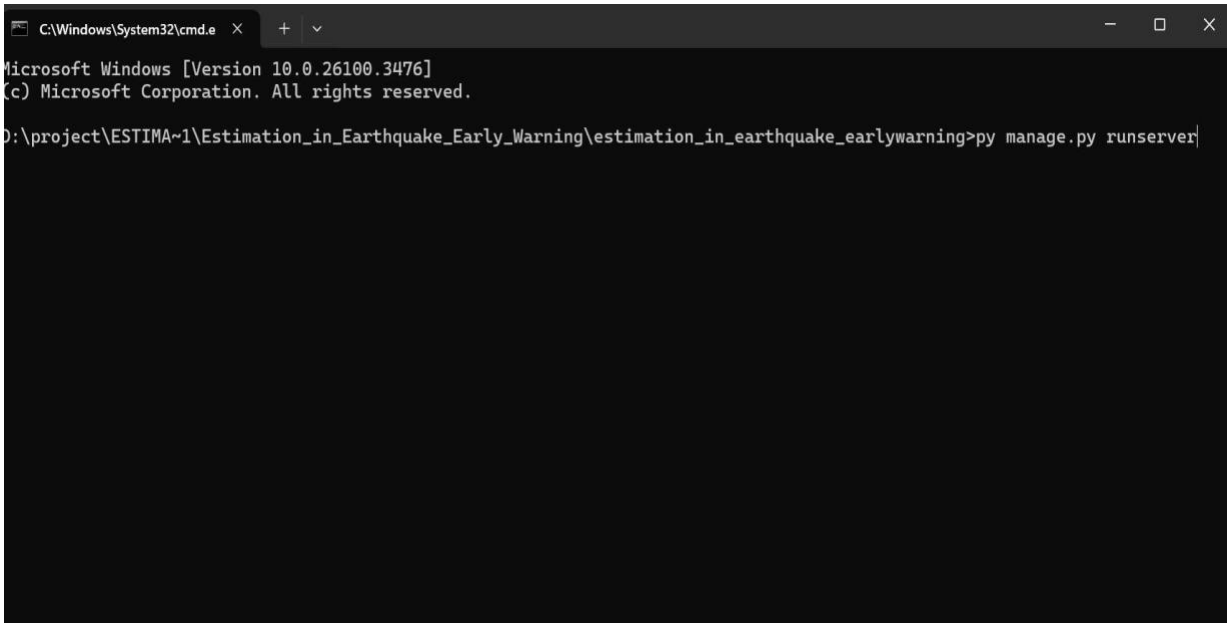


Fig 10.1: Starting command to run the Django application" python manage.py runserver".

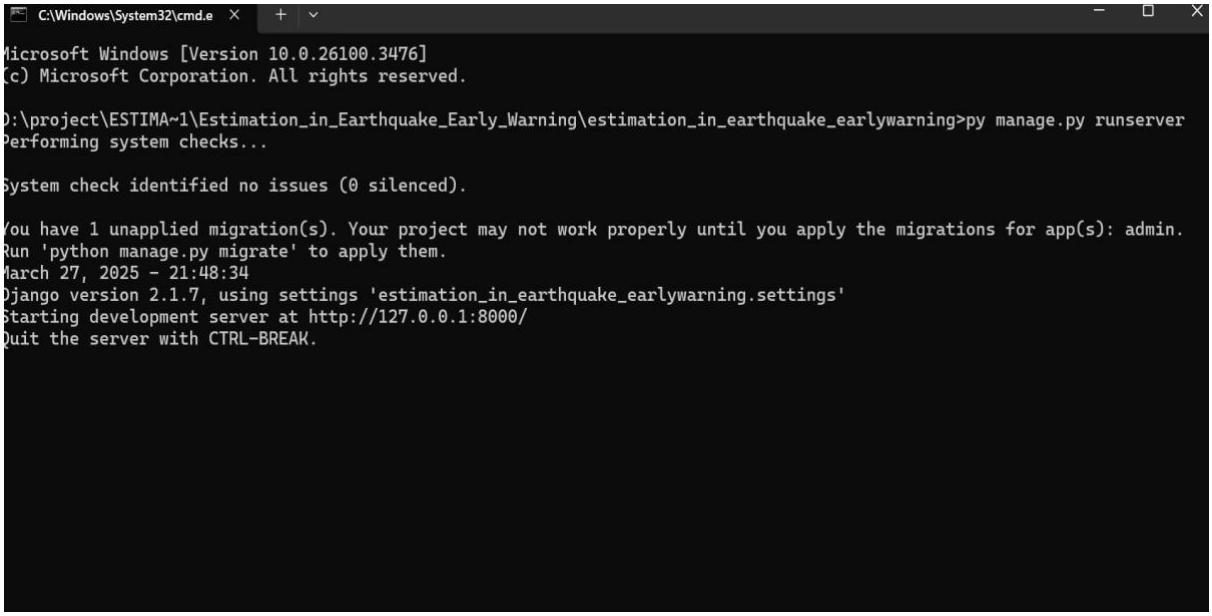


Fig 10.2: generation of Hyperlink.

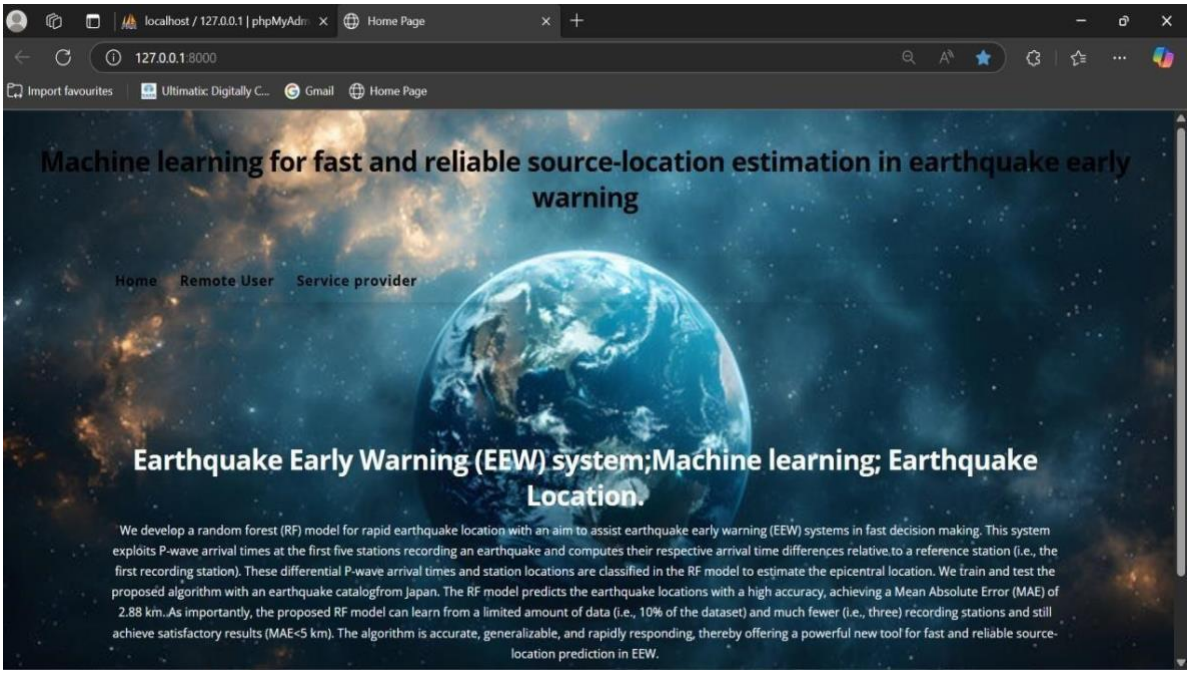


Fig 10.3: Home.html

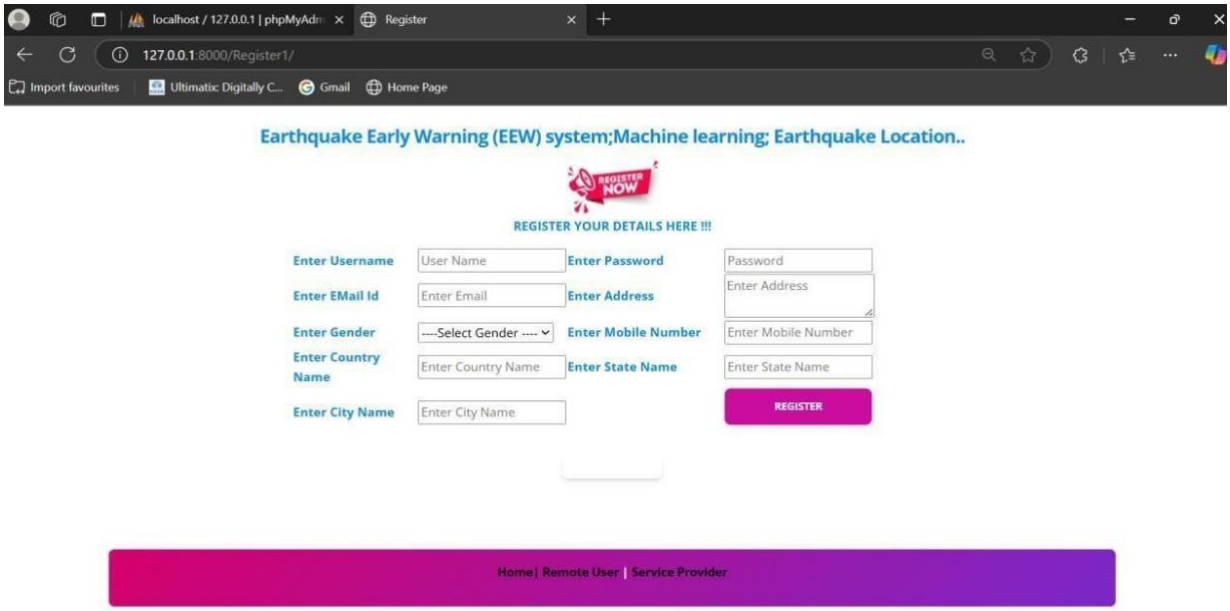


Fig 10.4: If You want to login in to the page new user must be registered .

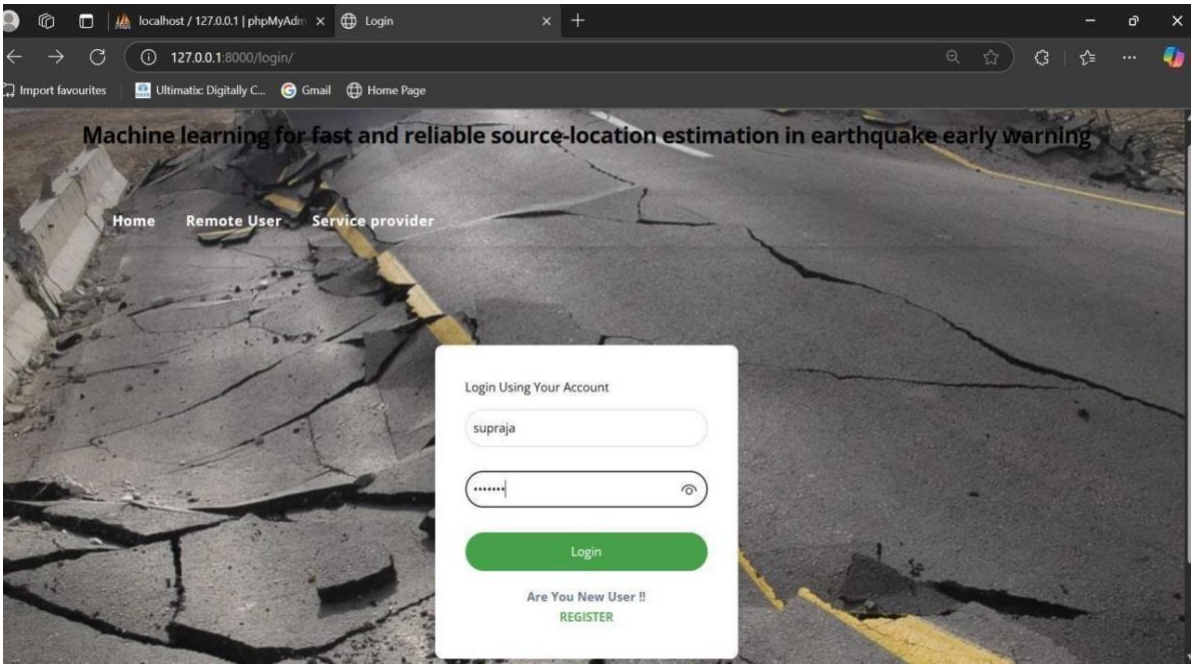


Fig 10.5: Login to remote user page

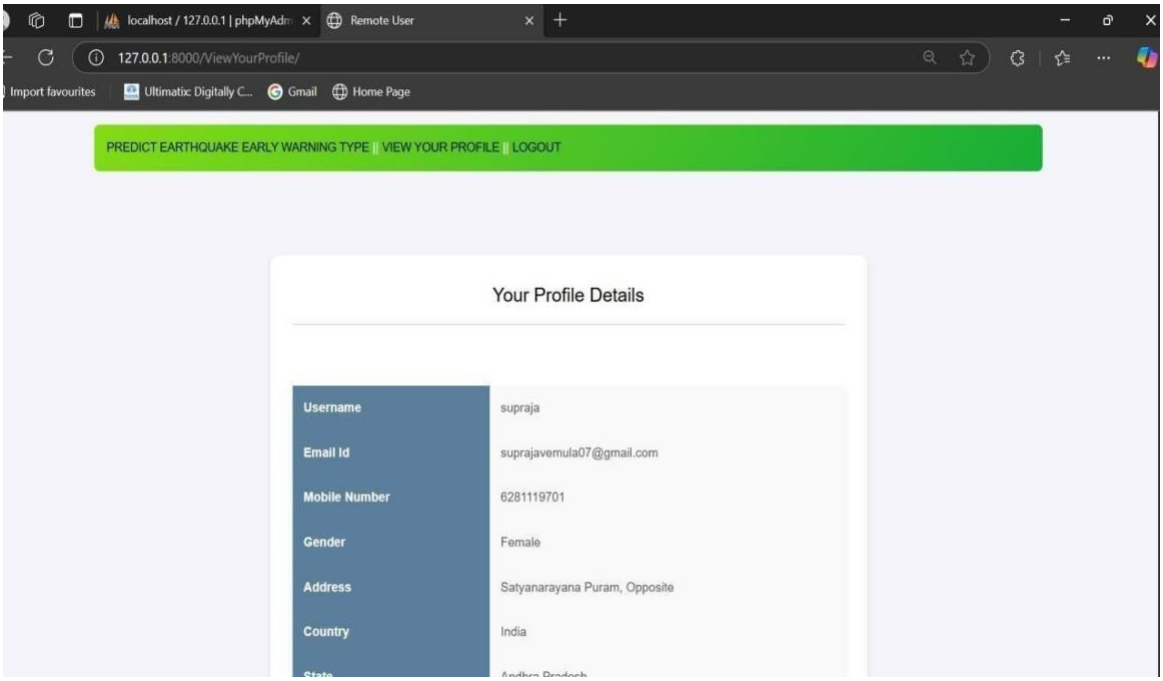


Fig 10.6: User can view their details in profile.

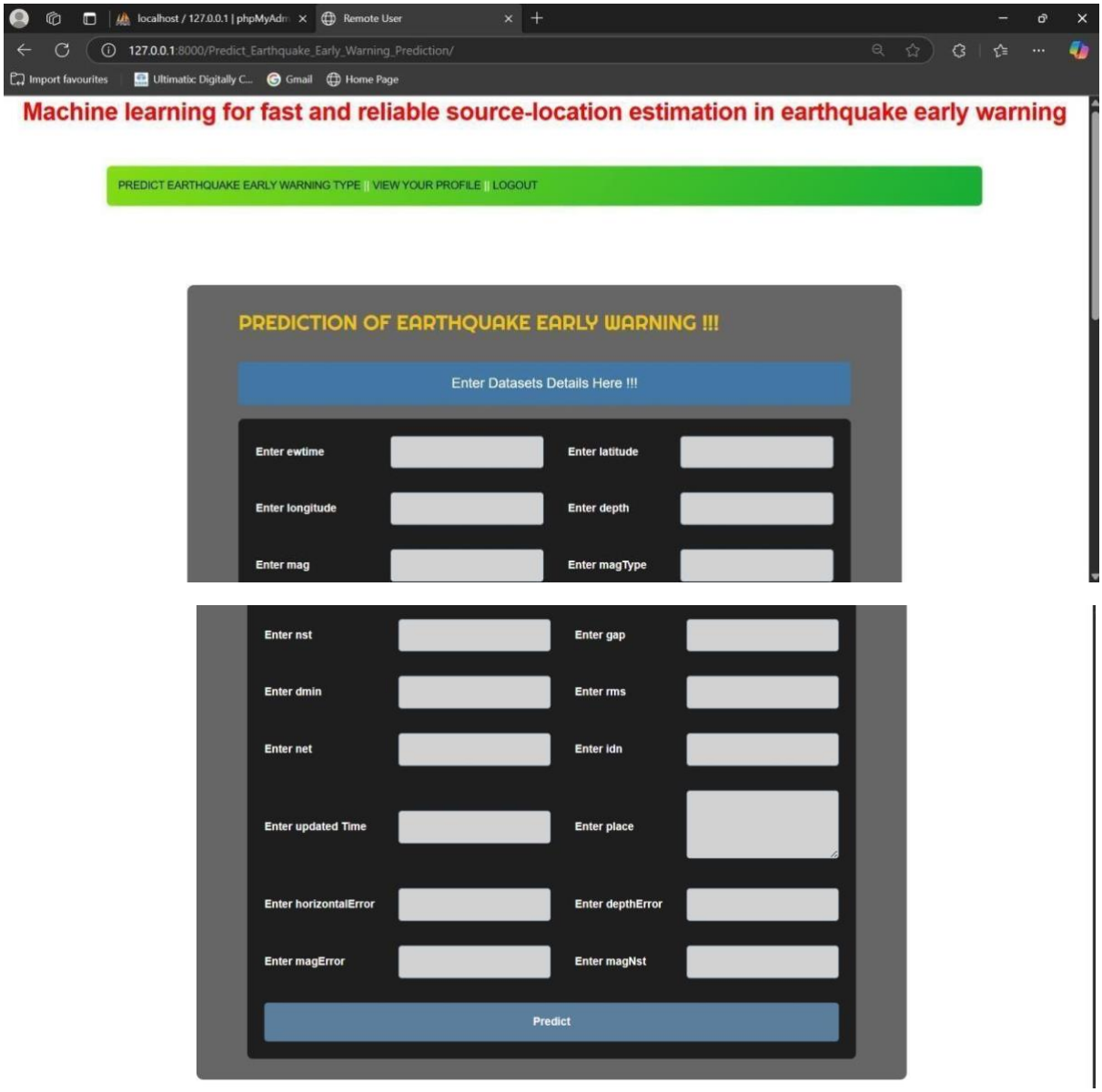


Fig 10.7 : Fill the details for prediction of early earthquake warning and then predict the output.

Fig 10.8 : Output will be predicted like this.

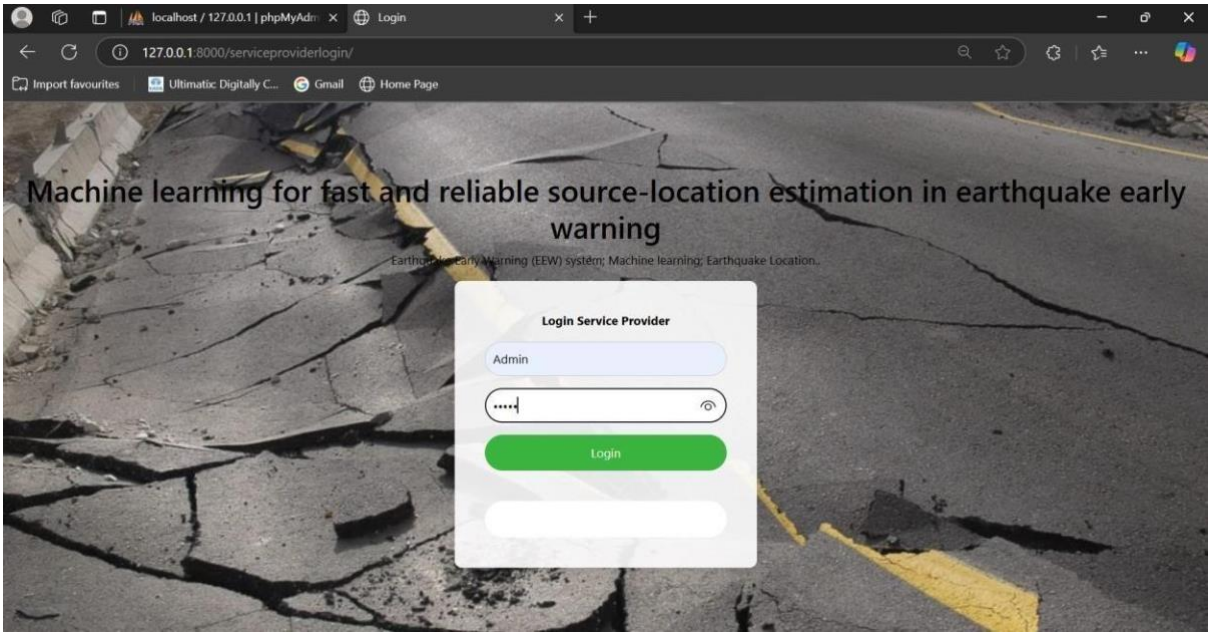


Fig 10.9 : Admin will be login into service provider page.

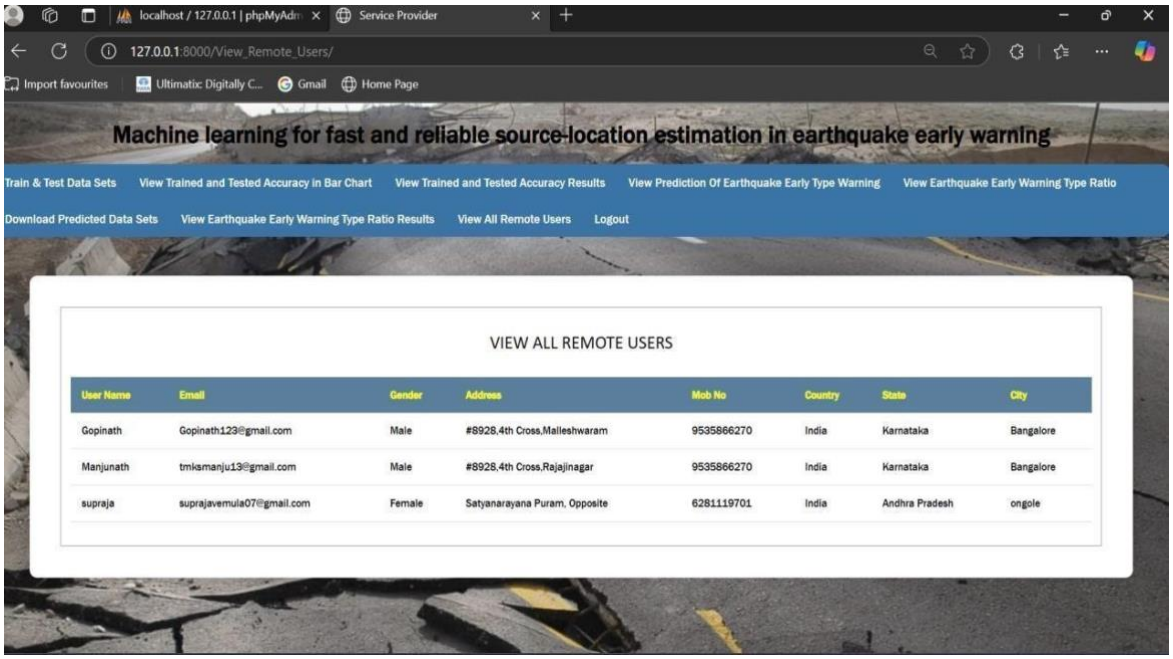


Fig 10.10 : we can view all remote user here.

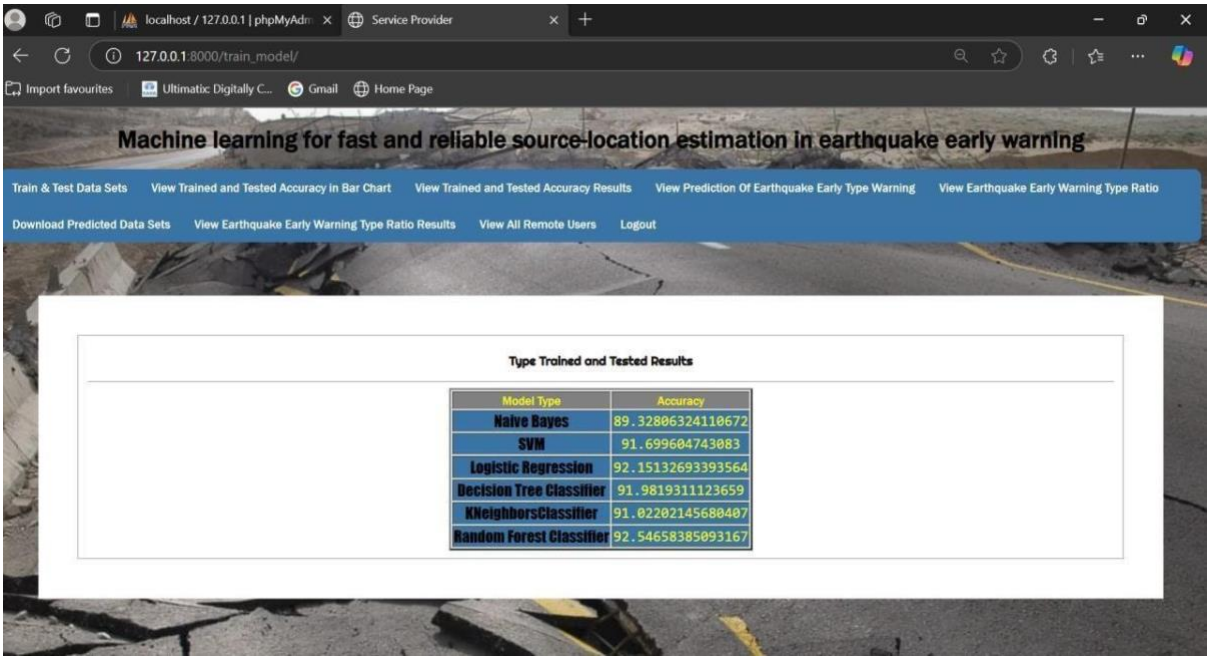


Fig 10.11 : Here dataset will be trained and tested.

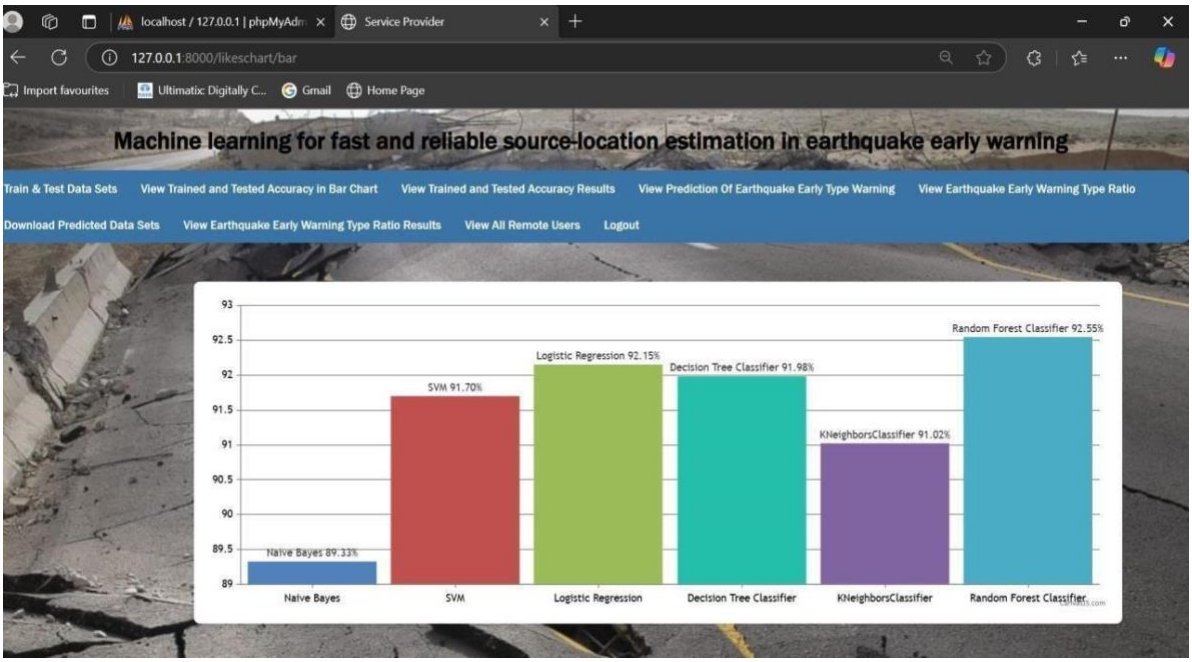


Fig 10.12 : Here dataset can be represented in a bar chart.

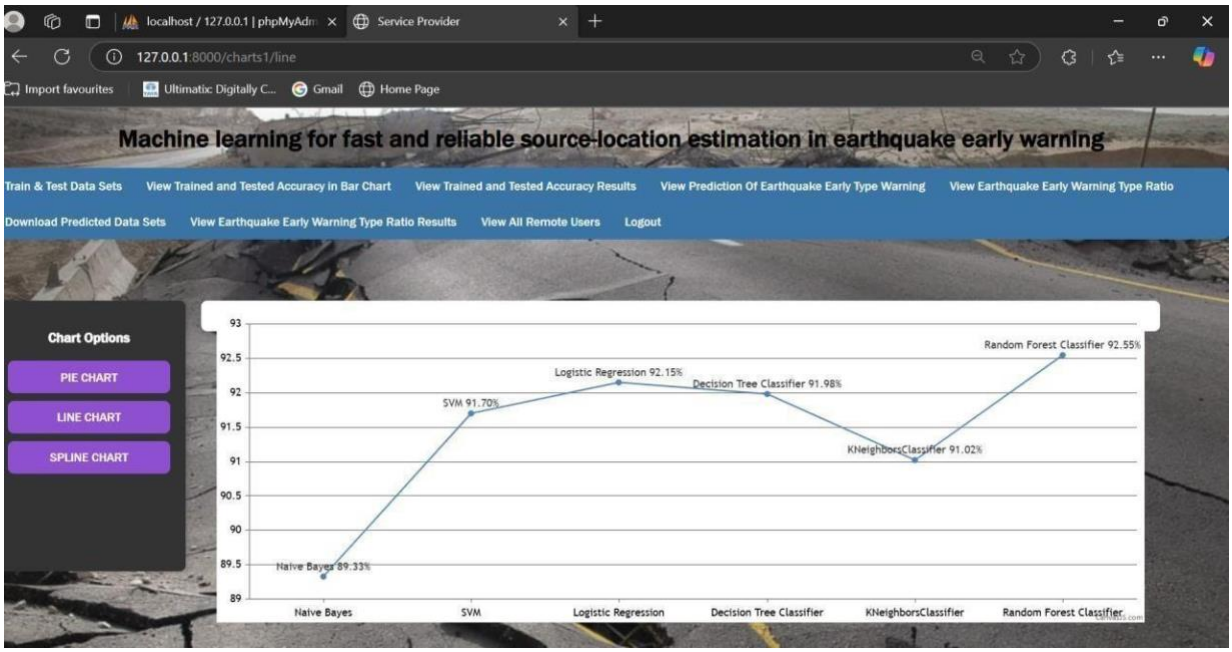


Fig 10.13 : Here data accuracy results will be represented in different chart options.

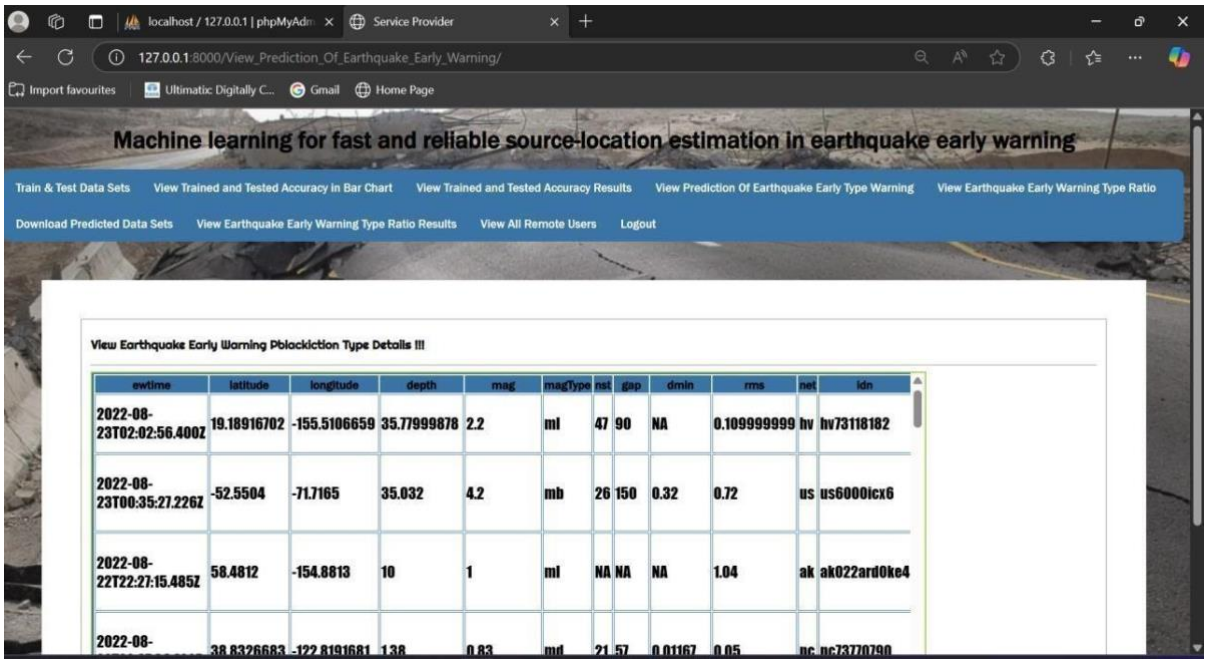


Fig 10.14 : Here you can view prediction of earthquake early type warning.

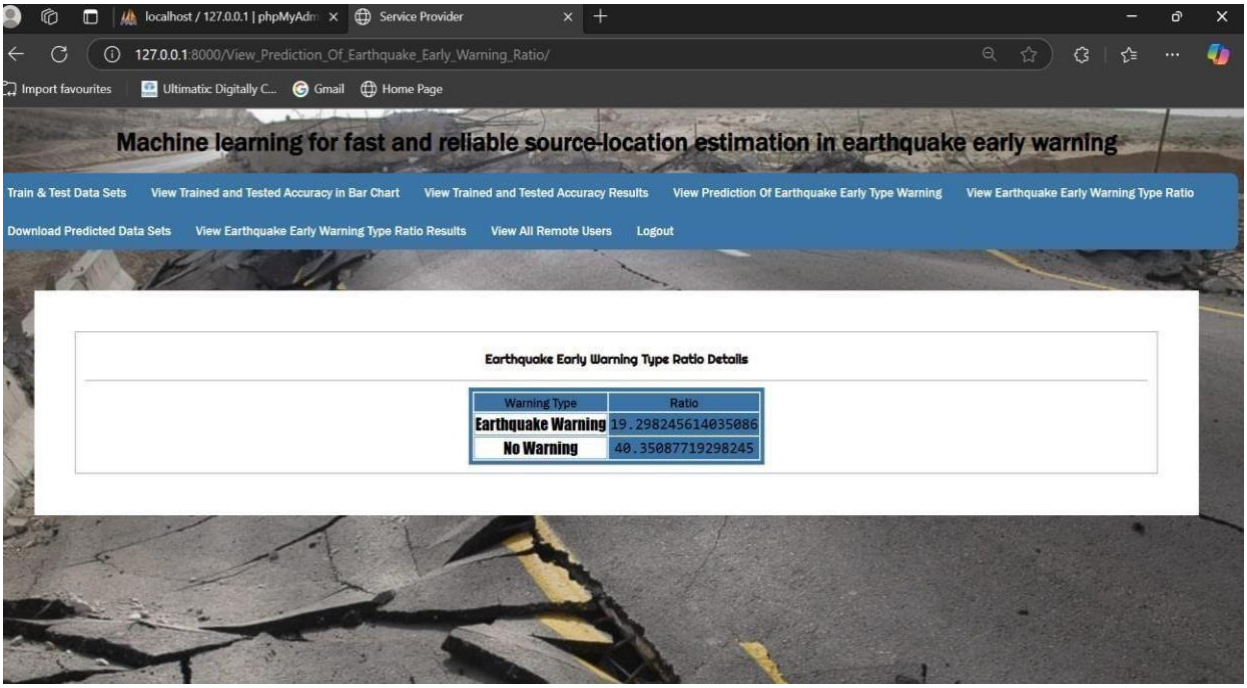


Fig 10.15 : view earthquake early warning type ratio.

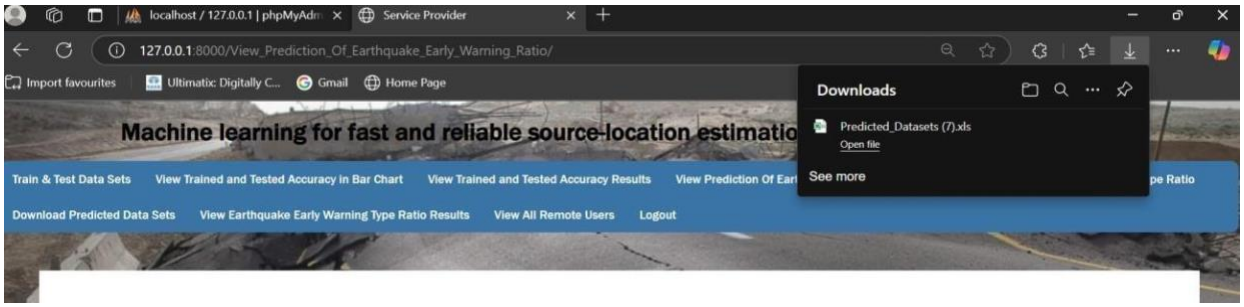


Fig 10.16 : Download datasets.

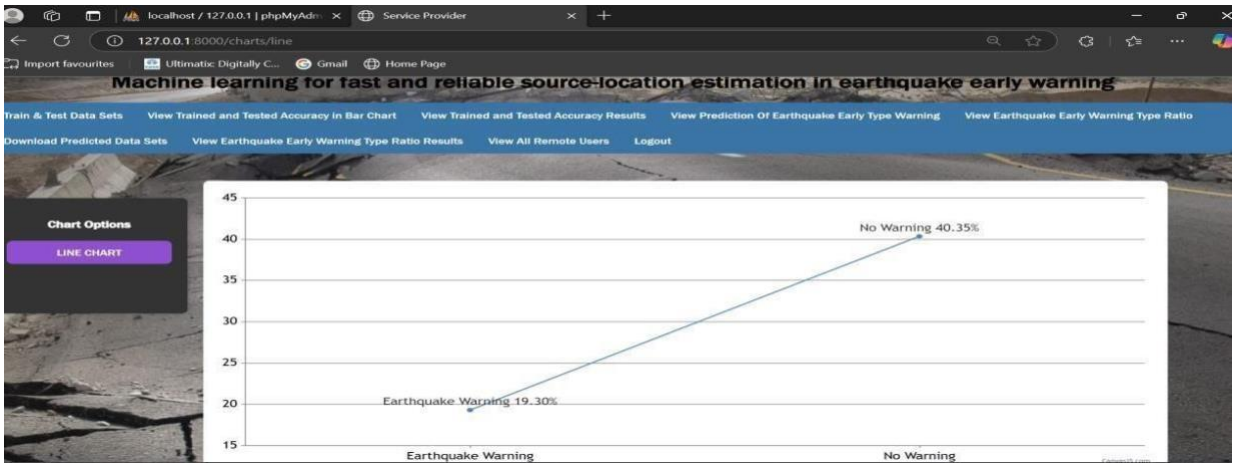


Fig 10.17 : Ratio results will be represented in chart form.

CHAPTER-11

CONCLUSION

CONCLUSION

The aim of this research is to improve the statistical fitness of the proposed model to overcome a KNN problem due to its computation approach. The KNN classifier can compute the empirical distribution over the Profit and Loss class values in the k number of nearest neighbors. However, the outcome is less than adequate due to sparse data. The KNN classifier has under fitting issue as it does not cater to generalization of sparse data outside the range of nearest neighborhood.

We have compared a hybrid KNN-Probabilistic model with four standard algorithms on the problem of predicting the stock price trends. Our results showed that the proposed KNN-Probabilistic model leads to significantly better results compared to the standard KNN algorithm and the other classification algorithms.

The limitation of the proposed model is that it applies a binary classification technique. The actual output of this binary classification model is a prediction score in two-class. The score indicates the model's certainty that the given observation belongs to either the Profit class or Loss class. For future work, the knowledge component is to transform the binary classification into multiclass classification. The multiclass classification involves observation and analysis of more than the existing two statistical class values. Additional research will include the application of the probabilistic model to multiclass data in order to provide more specific information of each class value. The newly formed multiclass classification will contain five class labels named “Sell”, “Underperform”, “Hold”, “Outperform”, and “Buy”. In numerical values for mapping purpose, we will convert “Sell” to -2 which implies strongly unfavorable; “Underperform” to -1 which implies moderately unfavorable; “Hold” to 0 which implies neutral; “Outperform” to 1 which implies moderately favorable; and “Buy” to 2 which implies strongly favorable.

CHAPTER-12
BIBLIOGRAPY

BIBLIOGRAPHY

REFERENCES

[1] Benjamin Graham, Jason Zweig, and Warren E. Buffett, The Intelligent Investor, Publisher: Harper Collins Publishers Inc, 2003.

[2] Charles D. Kirkpatrick II and Julie R. Dahlquist, Technical Analysis: The Complete Resource for Financial Market Technicians (3rd Edition), Pearson Education, Inc., 2015.

[3] Bruce Vanstone and Clarence Tan, A Survey of the Application of Soft Computing to Investment and Financial Trading, Proceedings of the Australian and New Zealand Intelligent

Information Systems Conference, Vol. 1, Issue 1, http://epublications.bond.edu.au/infotech_pubs/ 13/, 2003, pp. 211–216.

[4] Monica Tirea and Viorel Negru, Intelligent Stock Market Analysis System - A Fundamental and Macro-economical Analysis Approach, IEEE, 2014.

[5] Kian-Ping Lim, Chee-Wooi Hooy, Kwok-Boon Chang, and Robert Brooks, Foreign investors and stock price efficiency: Thresholds, underlying channels and investor heterogeneity, The North American Journal of Economics and Finance. Vol. 36, http://linkinghub.elsevier.com/retrieve/pii/S106 2940815001230, 2016, pp. 1–28.

[6] Lamartine Almeida Teixeira and Adriano Lorena Inácio de Oliveira, A method for automatic stock trading combining technical analysis and nearest neighbor classification, Expert Systems with Applications, http://linkinghub.elsevier.com/retrieve/pii/S095 7417410002149, 2010, pp. 6885–6890.

[7] Banshidhar Majhi, Hasan Shalabi, and Mowafak Fathi, FLANN Based Forecasting of S&P 500 Index. Information Technology Journal, Vol. 4, Issue 3, http://www.scialert.net/abstract/?doi=itj.2005.2 89.292, 2005, pp. 289–292.

[8] Ritanjali Majhi, G. Panda, and G. Sahoo, Development and performance evaluation of FLANN based model for forecasting of stock markets, Expert Systems with Applications,

Vol. 36, Issue 3, http://linkinghub.elsevier.com/retrieve/pii/S095 7417408005526, 2009, pp.

6800–6808.

[9] Tong-Seng Quah and Bobby Srinivasan, Improving returns on stock investment through neural network selection, Expert Systems with Applications, Vol. 17, Issue 4, http://linkinghub.elsevier.com/retrieve/pii/S095 741749900041X, 1999, pp. 295–301.

[10] Halbert White, Economic prediction using neural networks: the case of IBM daily stock returns, IEEE International Conference on Neural Networks, http://ieeexplore.ieee.org/document/23959/, IEEE, 1988, pp. 451–458.

[11] Yauheniya Shynkevicha, T.M. McGinnity, Sonya A. Coleman, and Ammar Belatreche, Forecasting movements of health-care stock prices based on different categories of news articles using multiple kernel learning, Decision Support Systems, Vol. 85, http://linkinghub.elsevier.com/retrieve/pii/S016 7923616300252, 2016, pp. 74–83.

[12] Han Lock Siew and Md Jan Nordin, Regression Techniques for the Prediction of Stock Price Trend, 2012 International Conference on Statistics in Science, Business and Engineering (ICSSBE), IEEE, 2012.

[13] Chi Ma, Junnan Liu, Hongyan Sun, and Haibin Jin, A hybrid financial time series model based on neural networks, IEEE, 2017.

[14] Lean Yu, Shouyang Wang, and Kin Keung Lai, Mining Stock Market Tendency Using GABased Support Vector Machines, Internet and Network Economics, http://link.springer.com/10.1007/11600930_33, 2005, pp. 336–345.

[15] Fu-Yuan Huang, Integration of an Improved Particle Swarm Algorithm and Fuzzy Neural Network for Shanghai Stock Market Prediction, 2008 Workshop on Power Electronics and Intelligent Transportation System.[Online].August 2008, IEEE, http://ieeexplore.ieee.org/document/4634852/, 2008, pp. 242–247.

[16] Ching-hsue Cheng, Tai-liang Chen, and Hiajong Teoh, Multiple-Period Modified Fuzzy Time-Series for Forecasting TAIEX, Fourth International Conference on Fuzzy Systems and

Knowledge Discovery (FSKD 2007), http://ieeexplore.ieee.org/document/4406190/, IEEE, 2007, pp. 2–6.

[17] Pei-Chann Chang, Chin-Yuan Fan, and ShihHsin Chen, Financial Time Series Data Forecasting by Wavelet and TSK Fuzzy Rule Based System, Fourth International Conference on Fuzzy Systems and Knowledge Discovery (FSKD 2007), http://ieeexplore.ieee.org/document/4406255/, IEEE, 2007, pp. 331–335.

[18] Lixin Yu and Yan-Qing Zhang, Evolutionary Fuzzy Neural Networks for Hybrid Financial Prediction, IEEE Transactions on Systems, Man and Cybernetics, Part C (Applications and Reviews), Vol. 35, Issue 2, http://ieeexplore.ieee.org/document/1424198/, IEEE, 2005, pp. 244–249.

[19] Chokri Slim, Neuro-fuzzy network based on extended Kalman filtering for financial time series, World Academy of Science, Engineering and Technology, Vol. 15, http://citeseerx.ist.psu.edu/viewdoc/download?d oi=10.1.1.193.4464&rep=rep1&type=pdf, 2006, pp. 134–139.

[20] Gérard Biau and Luc Devroye, Lectures on the Nearest Neighbor Method, Springer, 2015.

[21] Saed Sayad, K Nearest Neighbors - Classification, [Online]. 2017, http://www.saedsayad.com/k_nearest_neighbors .htm, 2017, [Accessed: 10 January 2017].

[22] Dan Morris, Bayes' Theorem: A Visual Introduction for Beginners, Blue Windmill Media, 2017.

[23] James V Stone, Bayes' Rule With R: A Tutorial Introduction to Bayesian Analysis, Sebtel Press, 2016.

[24] Marco Scutari and Jean-Baptiste Denis, Bayesian Networks: With Examples in R, CRC Press, 2014.

[25] Peter Bruce and Andrew Bruce, Practical Statistics for Data Scientists: 50 Essential Concepts, O'Reilly Media, 2017.

[26] Ciaran Walsh, Key Management Ratios, 4th Edition (Financial Times Series), Prentice

Hall, 2009.

[27] Seyed Enayatolah Alavi, Hasanali Sinaei, and Elham Afsharirad, Predict the Trend of Stock Prices Using Machine Learning Techniques, IAIEST, 2015.

[28] Lock Siew Han and Md Jan Nordin, Integrated Multiple Linear Regression-One Rule Classification Model for the Prediction of Stock Price Trend, Journal of Computer Sciences, Vol 13 (9), 2017, pp. 422-429.

[29] Abdolhossein Zameni and Othman Yong, Share Price Performance of Malaysian IPOs Around Lock-Up Expirations, Advanced Science Letters, Vol 23(9), 2017, pp. 8094-8102.

[30] Abdolhossein Zameni and Othman Yong, LockUp Expiry And Trading Volume Behaviour Of Malaysian Initial Public Offering's, International Journal of Economics and Financial Issues, Vol 6(3), 2016, pp. 12-21.

[31] Hani A.K. Ihlayyel, Nurfadhlina Mohd Sharef, Mohd Zakree Ahmed Nazri, and Azuraliza Abu Bakar, An Enhanced Feature Representation Based On Linear Regression Model For Stock Market Prediction, Intelligent Data Analysis, Vol 22(1), 2018, pp. 45-76.

[32] Lida Nikmanesh and Abu Hassan Shaari Mohd Nor, Macroeconomic Determinants of Stock Market Volatility: An Empirical Study of Malaysia and Indonesia, Asian Academy of Management Journal, Vol 21(1), 2016, pp. 161- 180.

[33] Luigi Troiano, Pravesh Kriplani, and Irene Díaz, Regression Driven F-Transform and Application to Smoothing of Financial Time Series, IEEE, 2017.